

ARCHITECTURE CLOUD

Job1

Tout d'abord nous allons installer Docker

-Lancer Localstack : **`docker run -d -p4566:4566 -p8080:8080 --name localstack localstack/localstack`**

-AWS CLI pip :

`python3 -m venv ~/aws-venv`
`source ~/aws-venv/bin/activate`
`pip install --upgrade pip`
`pip install awscli awscli-local`

-Ajouter au PATH permanent:

`echo 'alias aws-venv="source ~/aws-venv/bin/activate"' >> ~/.bashrc`
`echo 'export PATH="$HOME/aws-venv/bin:$PATH"' >> ~/.bashrc`
`source ~/.bashrc`

-Tester : **`awslocal --version`**

-Configurer LocalStack : **`awslocal configure`**

Access Key: test,

Secret: test,

Region: us-east-1,

Endpoint: <http://localhost:4566>

EC2: simulation d'instances

ELB: Load balancer: permet de répartir automatiquement le trafic entrant d'applications sur plusieurs cibles et appliances virtuelles dans une ou plusieurs zones de disponibilité.

Autoscaling: permet de maintenir la disponibilité des applications, et d'ajouter ou de supprimer automatiquement des instances EC2.

Vérifier que LocalStack tourne: **`sudo docker ps | grep localstack`**

```
anna@anna:~$ sudo docker ps | grep localstack
473c13acbbde    localstack/localstack-pro    "docker-entrypoint.sh"    53 minutes ago    Up 53 minutes (healthy)    127.0.0.1:443->443/tcp, 127.0.0.1:4510-4560->4510-4560/tcp, 53/tcp, 5678/tcp, 127.0.0.1:4566->4566/tcp    localstack-main
```

Commandes

````bash`

`# Créer un Security Group`

`awslocal ec2 create-security-group \`

`--group-name webapp-sg \`

`--description "Security group for web app"`

`# Autoriser le trafic HTTP`

`awslocal ec2 authorize-security-group-ingress \`

`--group-name webapp-sg \`

```

--protocol tcp \
--port 80 \
--cidr 0.0.0.0/0

Lancer une instance EC2 (simulation)
awslocal ec2 run-instances \
 --image-id ami-12345678 \
 --instance-type t2.micro \
 --security-groups webapp-sg \
 --user-data file:///userdata.sh

Créer un Load Balancer
awslocal elbv2 create-load-balancer \
 --name webapp-alb \
 --subnets subnet-12345 subnet-67890

```

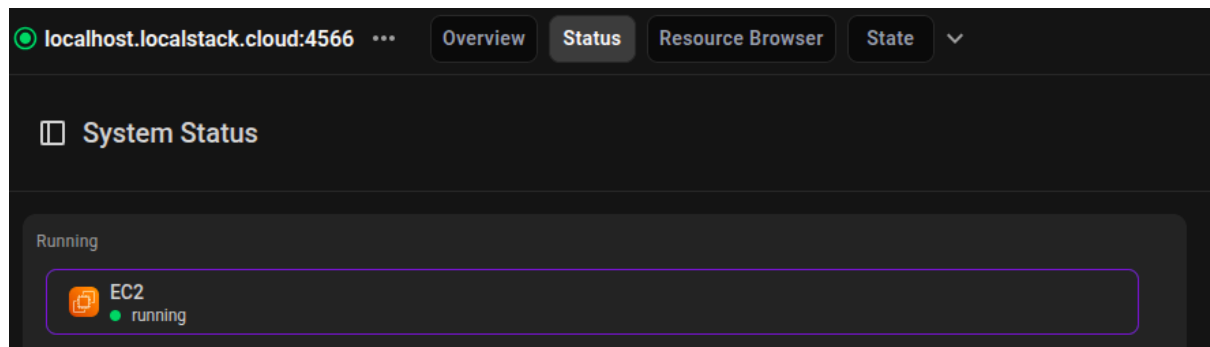
## 2) Création EC2

**awslocal ec2 create-security-group --group-name web-sg --description "Web"**

```

anna@anna:~$ awslocal ec2 create-security-group --group-name web-sg --description "Web"
{
 "GroupId": "sg-a874817c437603236",
 "SecurityGroupArn": "arn:aws:ec2:us-east-1:000000000000:security-group/sg-a874817c437603236"
}

```



Noter le GroupId: sg-a874817c437603236

```

anna@anna:~$ awslocal ec2 authorize-security-group-ingress --group-id sg-a874817c437603236 --protocol tcp --port 80 --cidr 0.0.0.0/0
{
 "Return": true,
 "SecurityGroupRules": [
 {
 "SecurityGroupRuleId": "sgr-1aba74698ccc82b83",
 "GroupId": "sg-a874817c437603236",
 "IsEgress": false,
 "IpProtocol": "tcp",
 "FromPort": 80,
 "ToPort": 80,
 "CidrIpv4": "0.0.0.0/0",
 "Tags": []
 }
]
}

```

Listez les AMI disponibles que l'on peut utiliser pour le projet:

**awslocal ec2 describe-images --owners amazon**

```

anna@anna:~$ awslocal ec2 describe-images --owners amazon
{
 "Images": [
 {
 "BlockDeviceMappings": [
 {
 "Ebs": {
 "DeleteOnTermination": false,
 "SnapshotId": "snap-7db17a2f3e4cedeb3",
 "VolumeSize": 15,
 "VolumeType": "standard"
 },
 "DeviceName": "/dev/sda1"
 }
],
 "Description": "Microsoft Windows Server 2016 Nano Locale English AMI provided by Amazon",
 "Hypervisor": "xen",
 "ImageOwnerAlias": "amazon",
 "Name": "Windows_Server-2016-English-Nano-Base-2017.10.13",
 "RootDeviceName": "/dev/sda1",
 "RootDeviceType": "ebs",
 "Tags": [],
 "VirtualizationType": "hvm",
 "BootMode": "uefi",
 "ImageId": "ami-03cf127a",
 "ImageLocation": "amazon/getting-started",
 "State": "available",
 "OwnerId": "801119661308",
 "CreationDate": "2026-01-14T11:06:41.000Z",
 "Public": true,
 "ProductCodes": [],
 "Architecture": "x86_64",
 "ImageType": "machine",
 "Platform": "windows"
 },
 {
 "BlockDeviceMappings": [
 {
 "Ebs": {
 "DeleteOnTermination": false,
 "SnapshotId": "snap-161e40b6540f3b6e5",
 "VolumeSize": 15,
 "VolumeType": "standard"
 },
 "DeviceName": "/dev/sda1"
 }
],
 "Description": "Microsoft Windows Server 2008 R2 SP1 Datacenter 64-bit Locale English Base AMI provided by Amazon",
 "Hypervisor": "xen",
 "ImageOwnerAlias": "amazon",
 "Name": "Windows_Server-2008-R2_SP1-English-64Bit-Base-2017.10.13",
 "RootDeviceName": "/dev/sda1",
 "RootDeviceType": "ebs",
 "Tags": [],
 "VirtualizationType": "hvm",

```

Lancer une instance avec une AMI valide:

```

awslocal ec2 run-instances --image-id ami-0c02fb55956c7d316 --instance-type t2.micro
--security-group-ids sg-0xxxxxxx --count 1 --tag-specifications
'ResourceType=instance,Tags=[{Key=Name,Value=web1}]'

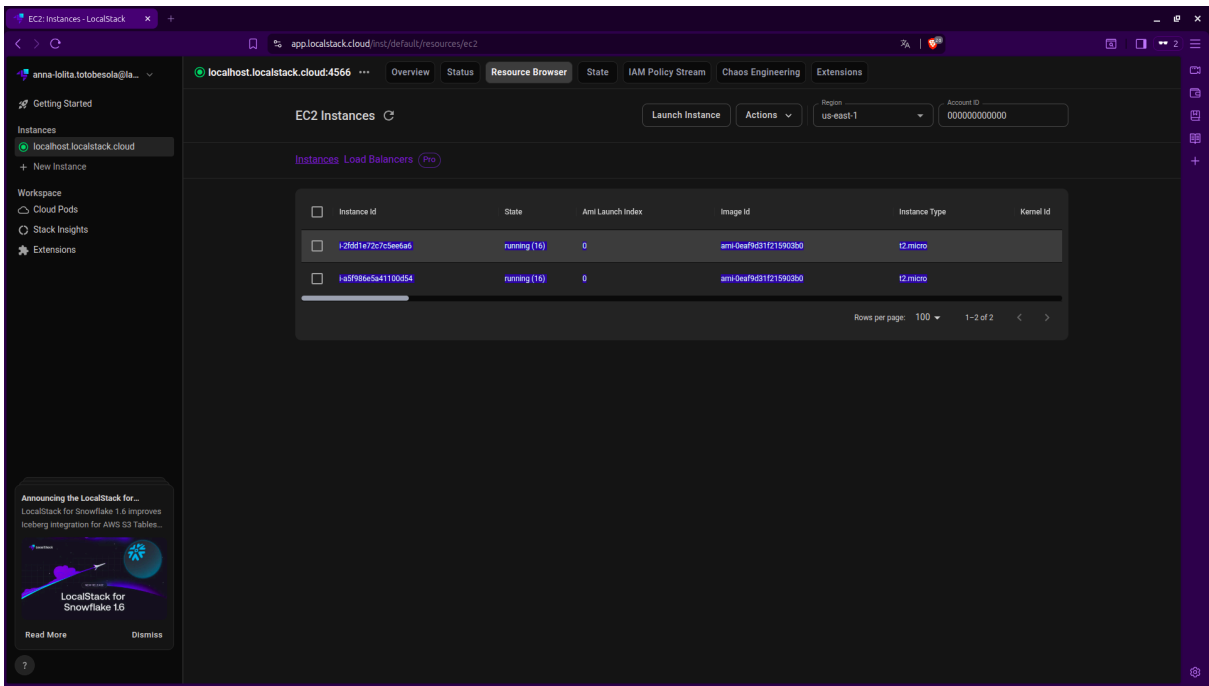
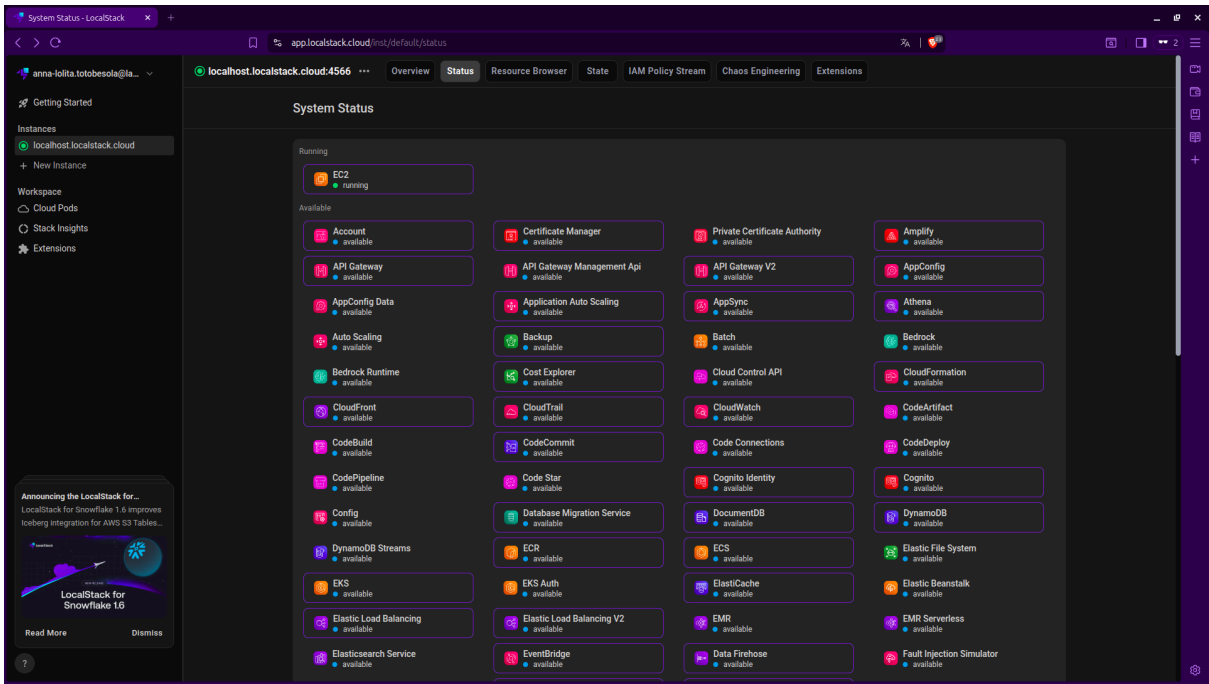
```

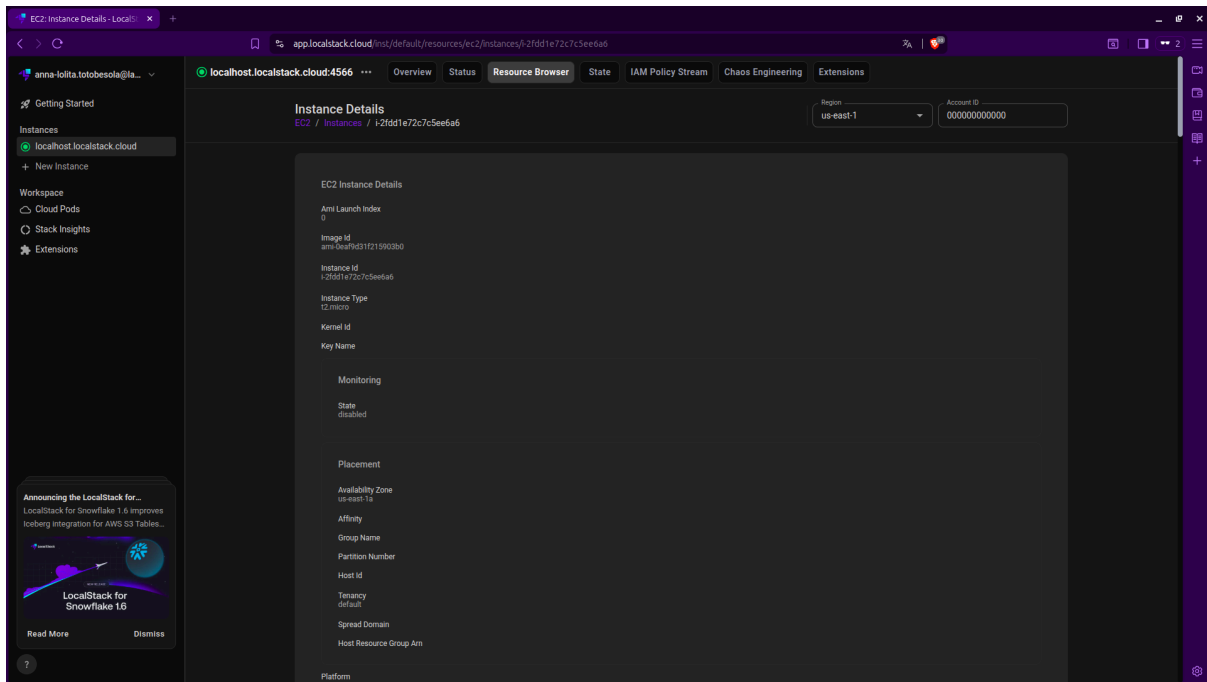
```
anna@anna: ~/Téléchargements
anna@anna:~$ awslocal ec2 run-instances --image-id ami-0ea9d31f215903b0 --instance-type t2.micro --security-group-ids sg-a874817c437603236 --count 1 --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=web1}]'
{
 "ReservationId": "r-6e9d44bb91c759b9",
 "OwnerId": "000000000000",
 "Groups": [],
 "Instances": [
 {
 "Architecture": "x86_64",
 "BlockDeviceMappings": [
 {
 "DeviceName": "/dev/sda1",
 "Ebs": {
 "AttachTime": "2026-01-14T11:41:23Z",
 "DeleteOnTermination": true,
 "Status": "attached",
 "VolumeId": "vol-31c5b741ceb674088"
 }
 }
],
 "ClientToken": "sf71m02-433f-4de4-8271-b978c0521850",
 "EbsOptimized": false,
 "Hypervisor": "xen",
 "NetworkInterfaces": [
 {
 "Association": {
 "IpOwnerId": "000000000000",
 "PublicDnsName": "ip-54-214-42-30.ec2.internal",
 "PublicIp": "54.214.42.30"
 },
 "Attachment": {
 "AttachTime": "2026-01-14T11:41:23Z",
 "AttachmentId": "eni-attach-11c7c9a663c06d45c",
 "DeleteOnTermination": false,
 "DeviceIndex": 0,
 "Status": "attached",
 "NetworkCardIndex": 0
 },
 "Description": "Primary network interface",
 "Groups": [
 {
 "GroupId": "sg-a874817c437603236",
 "GroupName": "web-sg"
 }
],
 "Ipv6Addresses": [],
 "MacAddress": "02:00:00:5302x:6802x:10002x",
 "NetworkInterfaceId": "eni-c13ea7dd51ecbeda6",
 "OwnerId": "000000000000",
 "PrivateDnsName": "ip-10-173-132-194.ec2.internal",
 "PrivateIpAddress": "10.173.132.194",
 "PrivateIpAddresses": [
 {
 "Association": {
 "IpOwnerId": "000000000000",
 "PublicDnsName": "ip-54-214-42-30.ec2.internal",
 "PublicIp": "54.214.42.30"
 }
 }
],
 "Status": "true"
 }
]
 }
]
}
```

Vérifier : ***awslocal ec2 describe-instances --filters "Name=tag:Name,Values=web1"***

```
anna@anna: ~/Téléchargements
anna@anna:~$ awslocal ec2 describe-instances --filters "Name=tag:Name,Values=web1"
{
 "Reservations": [
 {
 "ReservationId": "r-9386775b8a1d929c0",
 "OwnerId": "000000000000",
 "Groups": [],
 "Instances": [
 {
 "Architecture": "x86_64",
 "BlockDeviceMappings": [
 {
 "DeviceName": "/dev/sda1",
 "Ebs": {
 "AttachTime": "2026-01-14T11:40:09Z",
 "DeleteOnTermination": true,
 "Status": "attached",
 "VolumeId": "vol-c4b9300be5772628b"
 }
 }
],
 "ClientToken": "222400e6-ebb8-4fe7-b868-67655a914be9",
 "EbsOptimized": false,
 "Hypervisor": "xen",
 "NetworkInterfaces": [
 {
 "Association": {
 "IpOwnerId": "000000000000",
 "PublicDnsName": "ip-54-214-162-2.ec2.internal",
 "PublicIp": "54.214.162.2"
 },
 "Attachment": {
 "AttachTime": "2026-01-14T11:40:09Z",
 "AttachmentId": "eni-attach-7000f3394e7f13fd0",
 "DeleteOnTermination": false,
 "DeviceIndex": 0,
 "Status": "attached",
 "NetworkCardIndex": 0
 },
 "Description": "Primary network interface",
 "Groups": [
 {
 "GroupId": "sg-a874817c437603236",
 "GroupName": "web-sg"
 }
],
 "Ipv6Addresses": [],
 "MacAddress": "02:00:00:13902x:15802x:16802x",
 "NetworkInterfaceId": "eni-eba63d2d1cfe28680",
 "OwnerId": "000000000000",
 "PrivateDnsName": "ip-10-121-250-98.ec2.internal",
 "PrivateIpAddress": "10.121.250.98",
 "PrivateIpAddresses": [
 {
 "Association": {
 "IpOwnerId": "000000000000",
 "PublicDnsName": "ip-54-214-162-2.ec2.internal",
 "PublicIp": "54.214.162.2"
 }
 }
],
 "Status": "true"
 }
]
 }
]
 }
]
}
```

Dans LocalStack nous avons:





## Auto Scaling Group

Lister les ressources réseaux : ***awslocal ec2 describe-vpcs***

```
anna@anna:~$ awslocal ec2 describe-vpcs
{
 "Vpcs": [
 {
 "OwnerId": "000000000000",
 "InstanceTenancy": "default",
 "Ipv6CidrBlockAssociationSet": [],
 "CidrBlockAssociationSet": [
 {
 "AssociationId": "vpc-cidr-assoc-369552d188b34fafb",
 "CidrBlock": "172.31.0.0/16",
 "CidrBlockState": {
 "State": "associated"
 }
 }
],
 "IsDefault": true,
 "Tags": [],
 "VpcId": "vpc-b64641b5944905547",
 "State": "available",
 "CidrBlock": "172.31.0.0/16",
 "DhcpOptionsId": "default"
 }
]
}
```

***awslocal ec2 describe-subnets --filters "Name=vpc-id,Values=vpc-b64641b5944905547"***

```
anna@anna:~$ awslocal ec2 describe-subnets --filters "Name=vpc-id,Values=vpc-b64641b594905547"
{
 "Subnets": [
 {
 "AvailabilityZoneId": "us-east-1",
 "OwnerId": "000000000000",
 "AssignIpv6AddressesOnCreation": false,
 "Ipv6CidrBlockAssociationSet": [],
 "Tags": [],
 "SubnetArn": "arn:aws:ec2:us-east-1:000000000000:subnet/subnet-02aa201bb6150af8c",
 "Ipv6Native": false,
 "SubnetId": "subnet-02aa201bb6150af8c",
 "State": "available",
 "VpcId": "vpc-b64641b594905547",
 "CidrBlock": "172.31.0.0/20",
 "AvailableIpAddressCount": 4096,
 "AvailabilityZone": "us-east-1a",
 "DefaultForAz": true,
 "MapPublicIpOnLaunch": true
 },
 {
 "AvailabilityZoneId": "us-east-1",
 "OwnerId": "000000000000",
 "AssignIpv6AddressesOnCreation": false,
 "Ipv6CidrBlockAssociationSet": [],
 "Tags": [],
 "SubnetArn": "arn:aws:ec2:us-east-1:000000000000:subnet/subnet-df290b83599226e5a",
 "Ipv6Native": false,
 "SubnetId": "subnet-df290b83599226e5a",
 "State": "available",
 "VpcId": "vpc-b64641b594905547",
 "CidrBlock": "172.31.16.0/20",
 "AvailableIpAddressCount": 4096,
 "AvailabilityZone": "us-east-1b",
 "DefaultForAz": true,
 "MapPublicIpOnLaunch": true
 },
 {
 "AvailabilityZoneId": "us-east-1",
 "OwnerId": "000000000000",
 "AssignIpv6AddressesOnCreation": false,
 "Ipv6CidrBlockAssociationSet": [],
 "Tags": [],
 "SubnetArn": "arn:aws:ec2:us-east-1:000000000000:subnet/subnet-de29340d6c7735be1",
 "Ipv6Native": false,
 "SubnetId": "subnet-de29340d6c7735be1",
 "State": "available",
 "VpcId": "vpc-b64641b594905547",
 "CidrBlock": "172.31.32.0/20",
 "AvailableIpAddressCount": 4096,
 "AvailabilityZone": "us-east-1c",
 "DefaultForAz": true,
 "MapPublicIpOnLaunch": true
 },
 {
 "AvailabilityZoneId": "us-east-1",
 "OwnerId": "000000000000",
 "AssignIpv6AddressesOnCreation": false,
 "Ipv6CidrBlockAssociationSet": [],
 "Tags": [],
 "SubnetArn": "arn:aws:ec2:us-east-1:000000000000:subnet/subnet-de29340d6c7735be1",
 "Ipv6Native": false,
 "SubnetId": "subnet-de29340d6c7735be1",
 "State": "available",
 "VpcId": "vpc-b64641b594905547",
 "CidrBlock": "172.31.32.0/20",
 "AvailableIpAddressCount": 4096,
 "AvailabilityZone": "us-east-1c",
 "DefaultForAz": true,
 "MapPublicIpOnLaunch": true
 }
]
}
```

Créer Launch config

***awslocal autoscaling create-launch-configuration --launch-configuration-name web-lc --image-id ami-0c02fb55956c7d316 --instance-type t3.micro --security-groups sg-xxx***

```
anna@anna:~$ awslocal autoscaling create-launch-configuration --launch-configuration-name web-lc --image-id ami-0eaf9d31f215903b0 --instance-type t2.micro --security-groups sg-a874817c437603236
```

Création ASG

***awslocal autoscaling create-auto-scaling-group --auto-scaling-group-name web-asg --launch-configuration-name web-lc --min-size 1 --max-size 3 --desired-capacity 1 --vpc-zone-identifier subnet-abc --health-check-type EC2 (Remplacer VPC/Subnet/SG par vos propres valeurs )***

Pour vérifier ASG en CLI:

***awslocal autoscaling describe-auto-scaling-groups --auto-scaling-group-names web-asg***

```

anna@anna:~$ awslocal autoscaling create-auto-scaling-group --auto-scaling-group-name web-asg --launch-configuration-name web-lc --min-size 1 --max-size 3 --desired-capacity 1 --vpc-zone-identifier subnet-02aa201bb6150af8c --health-check-type EC2
anna@anna:~$ describe-subnets
describe-subnets : commande introuvable
anna@anna:~$ awslocal autoscaling describe-auto-scaling-groups --auto-scaling-group-names web-asg
{
 "AutoScalingGroups": [
 {
 "AutoScalingGroupName": "web-asg",
 "AutoScalingGroupARN": "arn:aws:autoscaling:us-east-1:000000000000:autoScalingGroup:cf7f84ea-1f0b-45f7-9d11-59425e3e22a7:autoScalingGroupName/web-asg",
 "LaunchConfigurationName": "web-lc",
 "MinSize": 1,
 "MaxSize": 3,
 "DesiredCapacity": 1,
 "DefaultCooldown": 300,
 "AvailabilityZones": [
 "us-east-1a"
],
 "LoadBalancerNames": [],
 "TargetGroupARNs": [],
 "HealthCheckType": "EC2",
 "HealthCheckGracePeriod": 300,
 "Instances": [
 {
 "InstanceId": "i-39d8fe85cfe9be208",
 "InstanceType": "t2.micro",
 "AvailabilityZone": "us-east-1a",
 "LifecycleState": "InService",
 "HealthStatus": "Healthy",
 "LaunchConfigurationName": "web-lc",
 "ProtectedFromScaleIn": false
 }
],
 "CreatedTime": "2026-01-14T13:43:18.917295Z",
 "SuspendedProcesses": [],
 "VPCZoneIdentifier": "subnet-02aa201bb6150af8c",
 "EnabledMetrics": [],
 "Tags": [],
 "TerminationPolicies": [
 "Default"
],
 "NewInstancesProtectedFromScaleIn": false,
 "ServiceLinkedRoleARN": "arn:aws:iam::000000000000:role/aws-service-role/autoscaling.amazonaws.com/AWSServiceRoleForAutoScaling",
 "CapacityRebalance": false
 }
]
}

```

Voir l'instance ASG

***awslocal ec2 describe-instances --instance-ids i-39d8fe85cfe9be208***

***awslocal autoscaling describe-auto-scaling-instances --instance-ids i-39d8fe85cfe9be208***

```

anna@anna:~$ awslocal ec2 describe-instances --instance-ids i-39d8fe85cfe9be208
awslocal autoscaling describe-auto-scaling-instances --instance-ids i-39d8fe85cfe9be208
{
 "Reservations": [
 {
 "ReservationId": "r-a6f6642b45302b121",
 "OwnerId": "000000000000",
 "Groups": [],
 "Instances": [
 {
 "Architecture": "x86_64",
 "BlockDeviceMappings": [
 {
 "DeviceName": "/dev/sda1",
 "Ebs": {
 "AttachTime": "2026-01-14T13:43:18Z",
 "DeleteOnTermination": true,
 "Status": "attached",
 "VolumeId": "vol-1ae1f3f0b9135fee6"
 }
 }
],
 "EbsOptimized": false,
 "Hypervisor": "xen",
 "NetworkInterfaces": [
 {
 "Association": {},
 "Attachment": {
 "AttachTime": "2026-01-14T13:43:18Z",
 "AttachmentId": "eni-attach-4e03b2fa29bee1bfe",
 "DeleteOnTermination": false,
 "DeviceIndex": 0,
 "Status": "attached",
 "NetworkCardIndex": 0
 },
 "Description": "Primary network interface",
 "Groups": [
 {
 "GroupId": "sg-a874817c437603236",
 "GroupName": "web-sg"
 }
],
 "Ipv6Addresses": [],
 "MacAddress": "02:00:00:4402x:17602x:902x",
 "NetworkInterfaceId": "eni-823a09ba046627c70",
 "OwnerId": "000000000000",
 "PrivateDnsName": "ip-172-31-0-4.ec2.internal",
 "PrivateIpAddress": "172.31.0.4",
 "PrivateIpAddresses": [
 {
 "Primary": true,
 "PrivateIpAddress": "172.31.0.4"
 }
],
 "SourceDestCheck": true,
 "Status": "in-use"
 }
]
 }
]
 }
]
}

```

localhost.localstack.cloud:4566 ...

Overview

Status

Resource Browser

State



## System Status

Running



Auto Scaling  
● running

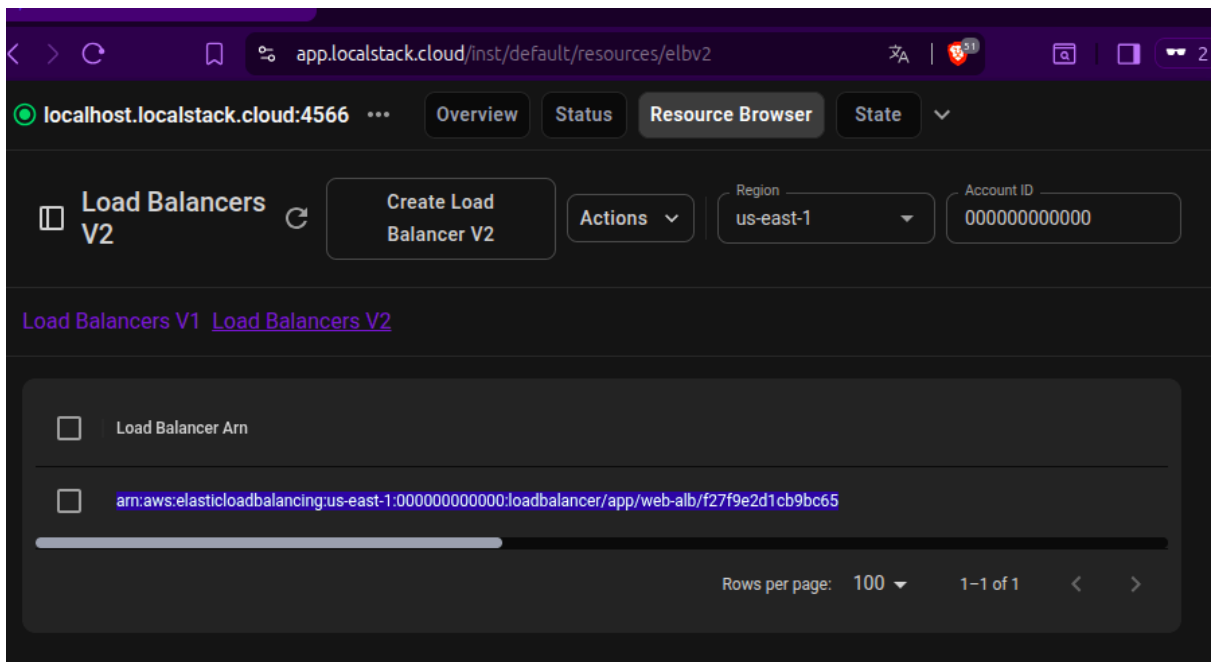
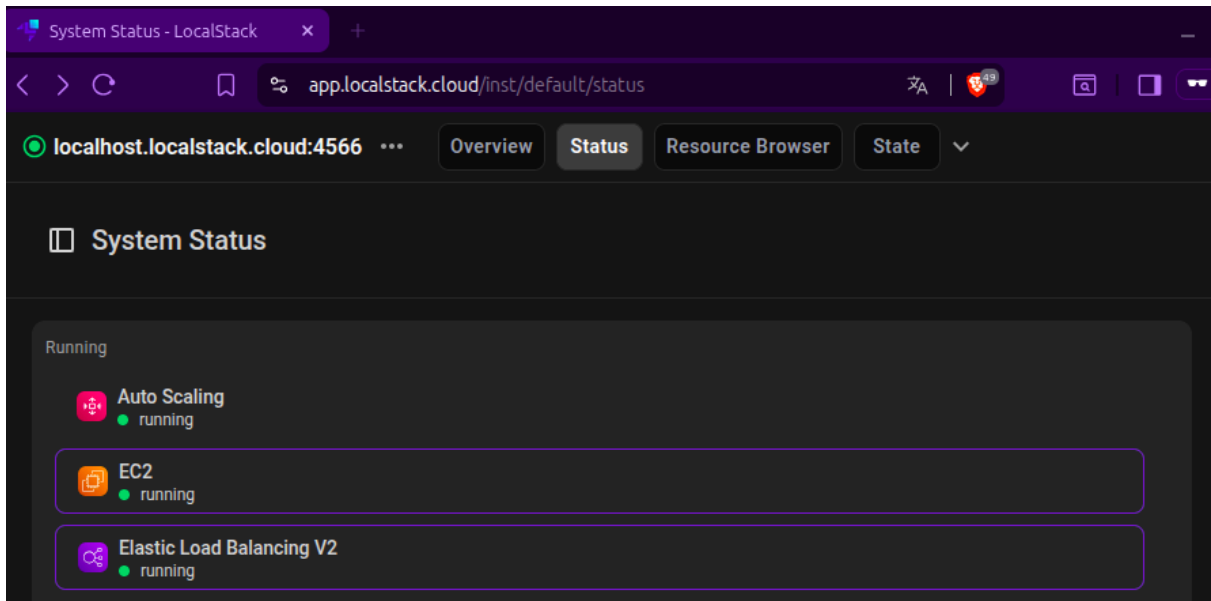


EC2  
● running

## Application Load Balancer (ALB)

```
anna@anna:~$ awslocal elbv2 describe-target-groups --names web-tg
{
 "TargetGroups": [
 {
 "TargetGroupArn": "arn:aws:elasticloadbalancing:us-east-1:000000000000:targetgroup/web-tg/537b031fd5be1ab5",
 "TargetGroupName": "web-tg",
 "Protocol": "HTTP",
 "Port": 80,
 "VpcId": "vpc-b64641b5944905547",
 "HealthCheckProtocol": "HTTP",
 "HealthCheckPort": "traffic-port",
 "HealthCheckEnabled": true,
 "HealthCheckIntervalSeconds": 30,
 "HealthCheckTimeoutSeconds": 5,
 "HealthyThresholdCount": 5,
 "UnhealthyThresholdCount": 2,
 "HealthCheckPath": "/",
 "Matcher": {
 "HttpCode": "200"
 },
 "LoadBalancerArns": [
 "arn:aws:elasticloadbalancing:us-east-1:000000000000:loadbalancer/app/web-alb/f27f9e2d1cb9bc65",
 "arn:aws:elasticloadbalancing:us-east-1:000000000000:loadbalancer/app/web-alb/f27f9e2d1cb9bc65",
 "arn:aws:elasticloadbalancing:us-east-1:000000000000:loadbalancer/app/web-alb/f27f9e2d1cb9bc65",
 "arn:aws:elasticloadbalancing:us-east-1:000000000000:loadbalancer/app/web-alb/f27f9e2d1cb9bc65"
],
 "TargetType": "instance",
 "ProtocolVersion": "HTTP1",
 "IpAddressType": "ipv4"
 }
]
}
```

```
anna@anna:~$ awslocal elbv2 register-targets --target-group-arn arn:aws:elasticloadbalancing:us-east-1:000000000000:targetgroup/web-tg/537b031fd5be1ab5 --targets Id=i-39d8fe85cfe9be208,Port=80
anna@anna:~$ awslocal elbv2 create-listener --load-balancer-arn arn:aws:elasticloadbalancing:us-east-1:000000000000:loadbalancer/app/web-alb/f27f9e2d1cb9bc65 --protocol HTTP --port 80 --default-actions Type=forward,TargetGroupArn=arn:aws:elasticloadbalancing:us-east-1:000000000000:targetgroup/web-tg/537b031fd5be1ab5
{
 "Listeners": [
 {
 "ListenerArn": "arn:aws:elasticloadbalancing:us-east-1:000000000000:listener/app/web-alb/f27f9e2d1cb9bc65/48693de49ef53705",
 "LoadBalancerArn": "arn:aws:elasticloadbalancing:us-east-1:000000000000:loadbalancer/app/web-alb/f27f9e2d1cb9bc65",
 "Port": 4566,
 "Protocol": "HTTP",
 "SslPolicy": "ELBSecurityPolicy-2016-08",
 "DefaultActions": [
 {
 "Type": "forward",
 "TargetGroupArn": "arn:aws:elasticloadbalancing:us-east-1:000000000000:targetgroup/web-tg/537b031fd5be1ab5"
 }
],
 "AlpnPolicy": []
 }
]
}
anna@anna:~$ awslocal elbv2 describe-target-health --target-group-arn arn:aws:elasticloadbalancing:us-east-1:000000000000:targetgroup/web-tg/537b031fd5be1ab5
curl web-alb.elb.localstack.cloud:4566
{
 "TargetHealthDescriptions": [
 {
 "Target": {
 "Id": "i-39d8fe85cfe9be208",
 "Port": 80
 },
 "HealthCheckPort": "traffic-port",
 "TargetHealth": {
 "State": "healthy"
 }
 }
]
}
```



## JOB2

*S3: stockage et hébergement web*

*CloudFront: accélère la distribution du contenu en acheminant chaque requête utilisateur*

*IAM: gestion d'identité*

#Créer un bucket S3

```
awslocal s3 mb s3://mon-site-web-static
```

#Configurer l'hébergement web

```
awslocal s3 website s3://mon-site-web-static \
```

```
--index-document index.html \
--error-document error.html
```

#Uploader les fichiers

```
echo '<h1>Mon Site Web</h1>' > index.html
awslocal s3 cp index.html s3://mon-site-web-static/
```

#Rendre le bucket public

```
awslocal s3api put-bucket-policy \
--bucket mon-site-web-static \
--policy file://bucket-policy.json
```

# Accéder au site

```
http://mon-site-web-static.s3.localstack.cloud:4566/index.html
```

...

**bucket-policy.json :**

```
```json  
{  
  "Version": "2012-10-17",  
  "Statement": [{  
    "Sid": "PublicReadGetObject",  
    "Effect": "Allow",  
    "Principal": "*",  
    "Action": "s3:GetObject",  
    "Resource": "arn:aws:s3:::mon-site-web-static/*"  
  }]  
}
```

```
anna@anna:~$ mkdir -p ~/static-site && cd ~/static-site  
cat > index.html << 'EOF'  
<!DOCTYPE html>  
<html>  
<head>  
<title>Job2 Static Site</title>  
</head>  
<body>  
<h1>Site Statique AWS S3+CloudFront</h1>  
<p>Job 2 réussi ! $(date)</p>  
</body>  
</html>  
EOF  
  
cat > error.html << 'EOF'  
<html>  
<body>  
<h1>404 Not Found</h1>  
</body>  
</html>  
EOF
```

```

anna@anna:~/static-site$ ls
error.html  index.html
anna@anna:~/static-site$ ls ~/static-site && cat ~/static-site/index.html
error.html  index.html
<!DOCTYPE html>
<html>
<head>
<title>Job2 Static Site</title>
</head>
<body>
<h1>Site Statique AWS S3+CloudFront</h1>
<p>Job 2 réussi ! $(date)</p>
</body>
</html>

```

```

anna@anna:~/static-site$ awslocal s3 mb s3://mon-site-static-job2
awslocal s3 website s3://mon-site-static-job2 --index-document index.html --error-document error.html
awslocal s3 sync ~/static-site/ s3://mon-site-static-job2/
make_bucket: mon-site-static-job2
upload: ./error.html to s3://mon-site-static-job2/error.html
upload: ./index.html to s3://mon-site-static-job2/index.html
anna@anna:~/static-site$ cat > ~/static-site/policy.json << 'EOF'
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "PublicReadGetObject",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::mon-site-static-job2/*"
  }]
}
EOF
awslocal s3api put-bucket-policy --bucket mon-site-static-job2 --policy file:///~/static-site/policy.json

```

```

anna@anna:~/static-site$ curl http://mon-site-static-job2.s3-website.localstack.cloud:4566
<!DOCTYPE html>
<html>
<head>
<title>Job2 Static Site</title>
</head>
<body>
<h1>Site Statique AWS S3+CloudFront</h1>
<p>Job 2 réussi ! $(date)</p>
</body>
</html>

```

app.localstack.cloud/inst/default/status

localhost.localstack.cloud:4566

Overview

Status

Resource Browser

State

System Status

Running

 Auto Scaling

● running

 EC2

● running

 Elastic Load Balancing V2

● running

 S3

● running

localhost.localstack.cloud:4566

Overview

Status

Resource Browser

State

Buckets

Create

Actions

Region

us-east-1

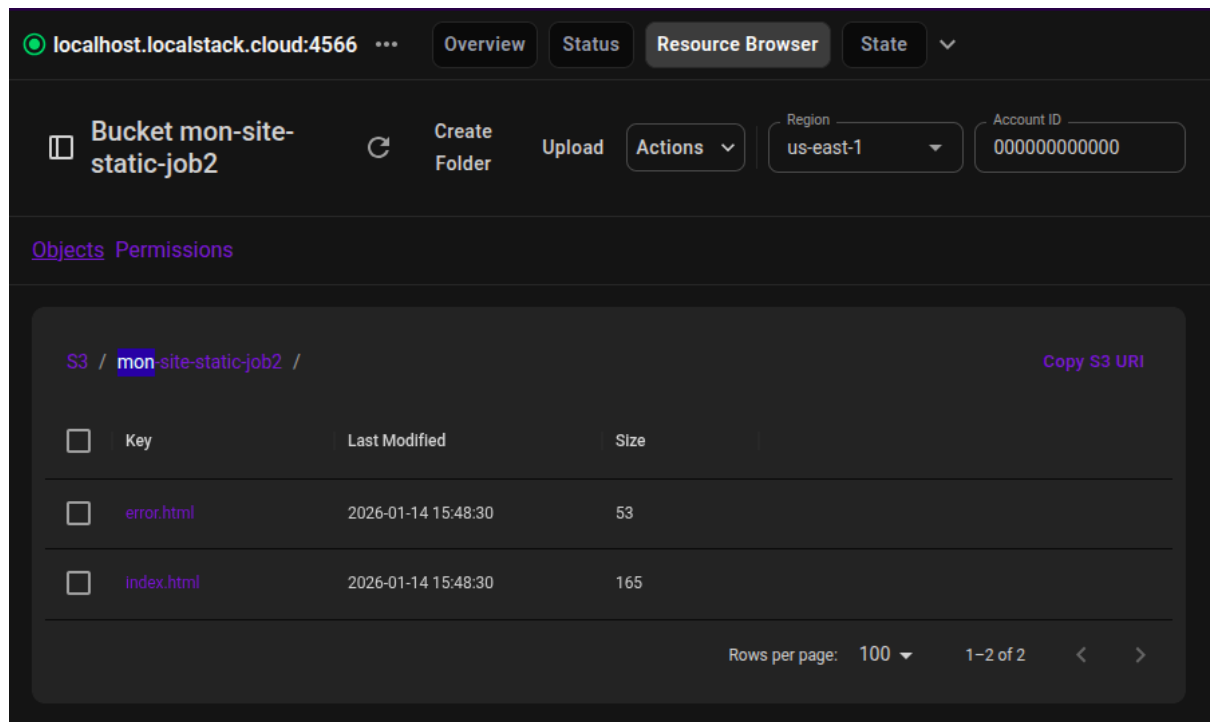
Account ID

000000000000

☐	Name	Creation Date
☐	mon-site-static-job2	2026-01-14 15:48:29

Rows per page: 100

1-1 of 1



Correction syntaxe Bash + Création répertoire/projet

text

```
mkdir -p ~/static-site && cd ~/static-site
```

- -p : crée sans erreur si existe déjà
- ~/static-site : dossier dédié Job 2

Génération 2 fichiers HTML essentiels

text

```
cat > index.html << 'EOF' # Page d'accueil
```

index.html : Page principale avec <h1>Site Statique AWS S3+CloudFront</h1>

- **\$(date) préservé**
- **Structure HTML5 valide**

error.html : Page 404 personnalisée

text

```
cat > error.html << 'EOF'
```

```
<html><body><h1>404 Not Found</h1></body></html>
```

Ensuite:

```
awslocal s3 mb s3://mon-site-static-job2
```

```
awslocal s3 website s3://mon-site-static-job2 --index-document index.html --error-document error.html
```

```
awslocal s3 sync ~/static-site/ s3://mon-site-static-job2/
```

```

anna@anna:~/static-site$ curl http://mon-site-static-job2.s3-website.localstack.cloud:4566
<!DOCTYPE html>
<html>
<head>
<title>Job2 Static Site</title>
</head>
<body>
<h1>Site Statique AWS S3+CloudFront</h1>
<p>Job 2 réussi ! $(date)</p>
</body>
</html>
anna@anna:~/static-site$ awslocal s3 mb s3://mon-site-static-job2
make_bucket: mon-site-static-job2
anna@anna:~/static-site$ awslocal s3 website s3://mon-site-static-job2 --index-document index.html --error-document error.html
anna@anna:~/static-site$ awslocal s3 sync ~/static-site/ s3://mon-site-static-job2/
upload: ./policy.json to s3://mon-site-static-job2/policy.json
anna@anna:~/static-site$ cat > ~/static-site/policy.json << 'EOF'
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "PublicReadGetObject",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::mon-site-static-job2/*"
  }]
}
EOF
awslocal s3api put-bucket-policy --bucket mon-site-static-job2 --policy file:///~/static-site/policy.json
anna@anna:~/static-site$ curl http://mon-site-static-job2.s3-website.localstack.cloud:4566
<!DOCTYPE html>
<html>
<head>
<title>Job2 Static Site</title>
</head>
<body>
<h1>Site Statique AWS S3+CloudFront</h1>
<p>Job 2 réussi ! $(date)</p>
</body>
</html>

```

localhost.localstack.cloud:4566
Overview
Status
Resource Browser
State

Bucket mon-site-static-job2
Create Folder
Upload
Actions
Region: us-east-1
Account ID: 000000000000

[Objects](#)
[Permissions](#)

S3 / mon-site-static-job2 /
Copy S3 URI

☐	Key	Last Modified	Size
☐	error.html	2026-01-14 15:48:30	53
☐	index.html	2026-01-14 15:48:30	165
☐	policy.json	2026-01-14 16:01:10	217

Rows per page: 100
1-3 of 3

Cloudfront

```
anna@anna:~/static-site$ awslocal cloudfront create-distribution --distribution-config '{
  "CallerReference": "job2-"$(date +%s)""",
  "Origins": {
    "Quantity": 1,
    "Items": [{
      "Id": "S3-mon-site-static-job2",
      "DomainName": "mon-site-static-job2.s3.localhost.localstack.cloud",
      "CustomOriginConfig": {
        "HTTPPort": 80,
        "HTTPSPort": 443,
        "OriginProtocolPolicy": "http-only"
      }
    }]
  },
  "DefaultCacheBehavior": {
    "TargetOriginId": "S3-mon-site-static-job2",
    "ViewerProtocolPolicy": "allow-all",
    "ForwardedValues": {
      "QueryString": false,
      "Cookies": {"Forward": "none"}
    }
  },
  "Enabled": true,
  "Comment": "Job2 Static CDN"
}'
{
  "Location": "http://localhost.localstack.cloud:4566/2020-05-31/distribution/660a13c8",
  "ETag": "6d53335c",
  "Distribution": {
    "Id": "660a13c8",
    "ARN": "arn:aws:cloudfront::000000000000:distribution/660a13c8",
    "Status": "InProgress",
    "LastModifiedTime": "2026-01-14T15:07:57.604000Z",
    "InProgressInvalidationBatches": 0,
    "DomainName": "660a13c8.cloudfront.localhost.localstack.cloud",
    "ActiveTrustedSigners": {
      "Enabled": false,
      "Quantity": 0
    },
    "ActiveTrustedKeyGroups": {
      "Enabled": false,
      "Quantity": 0
    },
    "DistributionConfig": {
      "CallerReference": "job2-1768403276",
      "Aliases": {
        "Quantity": 0
      },
      "DefaultRootObject": "",
      "Origins": {
        "Quantity": 1,
        "Items": [
          {
            "Id": "S3-mon-site-static-job2",
            "DomainName": "mon-site-static-job2.s3.localhost.localstack.cloud"
```

```

anna@anna:~/static-site$ awslocal cloudfront list-distributions
{
  "DistributionList": {
    "Items": [
      {
        "Id": "660a13c8",
        "ARN": "arn:aws:cloudfront::000000000000:distribution/660a13c8",
        "LastModifiedTime": "2026-01-14T15:07:57.604000Z",
        "DomainName": "660a13c8.cloudfront.localstack.cloud",
        "Aliases": {
          "Quantity": 0
        },
        "Origins": {
          "Quantity": 1,
          "Items": [
            {
              "Id": "S3-mon-site-static-job2",
              "DomainName": "mon-site-static-job2.s3.localstack.cloud",
              "OriginPath": "",
              "CustomHeaders": {
                "Quantity": 0
              },
              "CustomOriginConfig": {
                "HTTPPort": 80,
                "HTTPSPort": 443,
                "OriginProtocolPolicy": "http-only",
                "OriginReadTimeout": 30,
                "OriginKeepaliveTimeout": 5
              },
              "ConnectionAttempts": 3,
              "ConnectionTimeout": 10,
              "OriginShield": {
                "Enabled": false
              },
              "OriginAccessControlId": ""
            }
          ]
        },
        "OriginGroups": {
          "Quantity": 0
        },
        "DefaultCacheBehavior": {
          "TargetOriginId": "S3-mon-site-static-job2",
          "TrustedSigners": {
            "Enabled": false,
            "Quantity": 0
          },
          "TrustedKeyGroups": {
            "Enabled": false,
            "Quantity": 0
          },
          "ViewerProtocolPolicy": "allow-all",
          "AllowedMethods": {
            "Quantity": 2,
            "Items": [
              "HEAD",
              "GET"
            ]
          }
        }
      }
    ]
  }
}

```

localhost.localstack.cloud:4566

OverviewStatusResource BrowserState

System Status

Running

 Auto Scaling
● running

 CloudFront
● running

 EC2
● running

 Elastic Load Balancing V2
● running

 S3
● running

localhost.localstack.cloud:4566

OverviewStatusResource BrowserState

CloudFront Distributions

Create Distribution

Actions

Region
us-east-1

Account ID
000000000000

Id	ARN	Status
660a13c8	arn:aws:cloudfront::000000000000:distribution/660a13c8	

Rows per page: 100 1-1 of 1

localhost.localstack.cloud:4566 ... Overview Status **Resource Browser** State ▾

 **Distribution Details**
Distributions / 660a13c8 Edit Distribution Region: us-east-1 Account ID: 000000000000

Distribution Details

Id
660a13c8

ARN
arn:aws:cloudfront::000000000000:distribution/660a13c8

Status
Deployed

In Progress Invalidation Batches
0

Domain Name
660a13c8.cloudfront.localhost.localstack.cloud

Active Trusted Signers

Enabled
false

Quantity
0

Items

Active Trusted Key Groups

Enabled
false

Quantity
0

Items

```
anna@anna:~/static-site$ echo "CDN: 660a13c8.cloudfront.localhost.localstack.cloud"
CDN: 660a13c8.cloudfront.localhost.localstack.cloud
```

IAM : Création user pour un site web

awslocal iam create-user --user-name site-static-user

```
anna@anna:~$ awslocal iam create-user --user-name site-static-user
{
  "User": {
    "Path": "/",
    "UserName": "site-static-user",
    "UserId": "sapginmm6au5qkekzjne",
    "Arn": "arn:aws:iam::000000000000:user/site-static-user",
    "CreateDate": "2026-01-17T23:13:28.161852Z"
  }
}
```

```
anna@anna:~$ cat > s3-public-policy.json << EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::mon-site-static-laplateforme/*"
    }
  ]
}
EOF
```

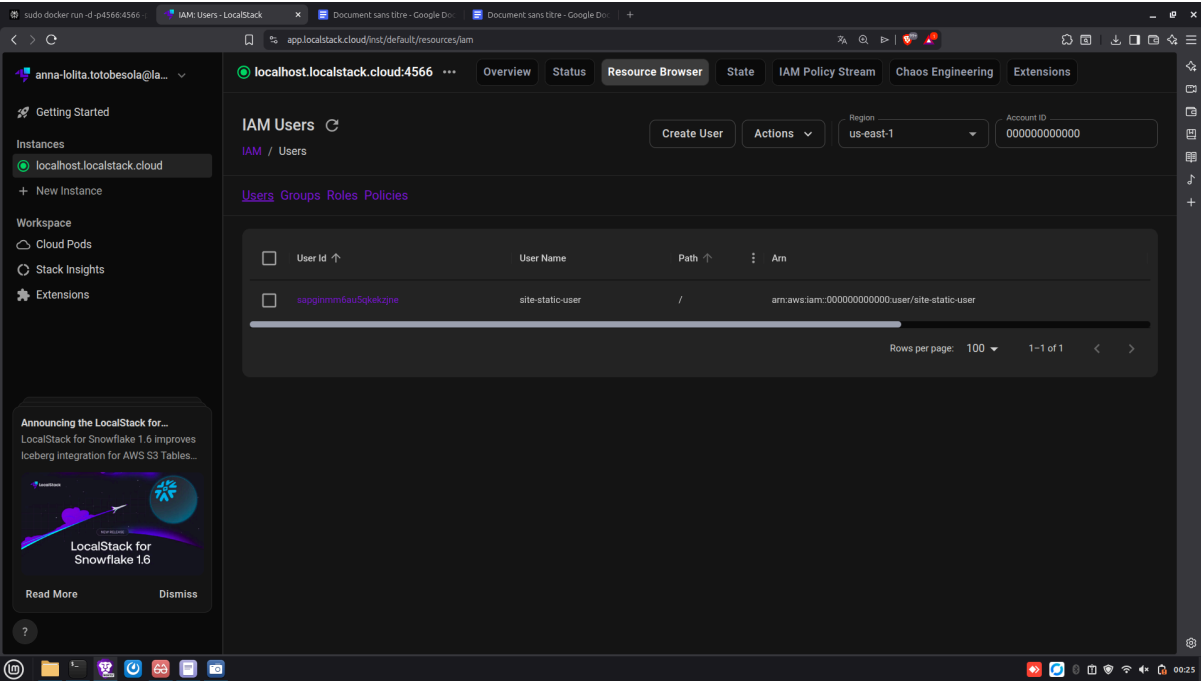
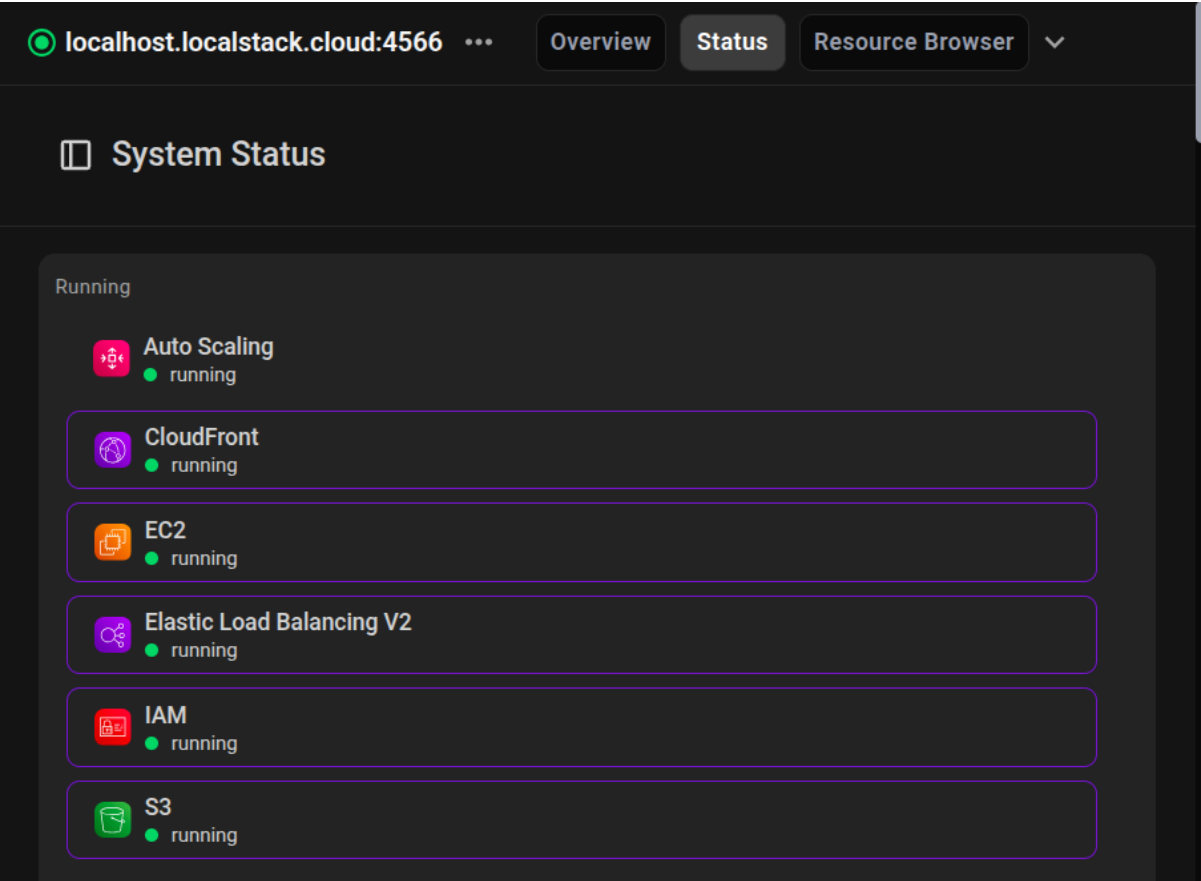
```
anna@anna:~$ awslocal s3api put-bucket-policy --bucket mon-site-static-laplateforme --policy file://s3-public-policy.json
```

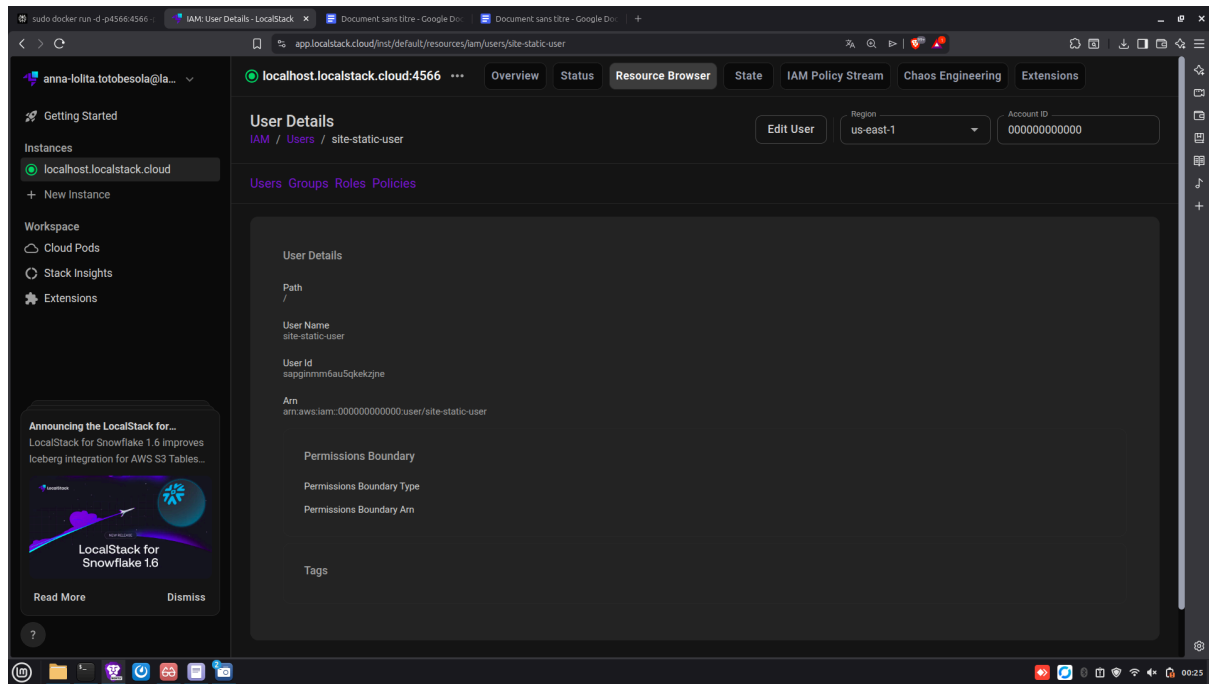
```
anna@anna:~$ cat > s3-upload-policy.json << EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:ListBucket"],
      "Resource": ["arn:aws:s3:::mon-site-static-laplateforme", "arn:aws:s3:::mon-site-static-laplateforme/*"]
    }
  ]
}
EOF

awslocal iam put-user-policy \
  --user-name site-static-user \
  --policy-name S3StaticUpload \
  --policy-document file:///s3-upload-policy.json
anna@anna:~$ curl "http://mon-site-static-laplateforme.s3-website.localhost.localstack.cloud:4566/"
anna@anna:~$ awslocal iam list-users
{
  "Users": [
    {
      "Path": "/",
      "UserName": "site-static-user",
      "UserId": "sapginmm6au5qkekzjne",
      "Arn": "arn:aws:iam::000000000000:user/site-static-user",
      "CreateDate": "2026-01-17T23:13:28.161852Z"
    }
  ]
}
```

```
anna@anna:~$ cat > s3-upload-policy.json << EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:ListBucket"],
      "Resource": ["arn:aws:s3:::mon-site-static-laplateforme", "arn:aws
:s3:::mon-site-static-laplateforme/*"]
    }
  ]
}
EOF

awslocal iam put-user-policy \
  --user-name site-static-user \
  --policy-name S3StaticUpload \
  --policy-document file:///s3-upload-policy.json
anna@anna:~$ curl "http://mon-site-static-laplateforme.s3-website.localh
ost.localstack.cloud:4566/"
anna@anna:~$ awslocal iam list-users
{
  "Users": [
    {
      "Path": "/",
      "UserName": "site-static-user",
      "UserId": "sapginmm6au5qkekzjne",
      "Arn": "arn:aws:iam::000000000000:user/site-static-user",
      "CreateDate": "2026-01-17T23:13:28.161852Z"
    }
  ]
}
```





JOB3 : Base de données RDS

RDS: service de base de données relationnelle

- Créer une table
- Insérer des données
- Lire les données

```
# Créer une table DynamoDB
awslocal dynamodb create-table \
  --table-name Orders \
  --attribute-definitions \
    AttributeName=OrderId,AttributeType=S \
  --key-schema \
    AttributeName=OrderId,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST
```

```
# Insérer des données
awslocal dynamodb put-item \
  --table-name Orders \
  --item '{
    "OrderId": {"S": "ORDER-001"},
    "Customer": {"S": "Alice"},
    "Total": {"N": "99.99"}
  }'
```

```
# Lire les données
awslocal dynamodb get-item \
  --table-name Orders \
  --key '{"OrderId": {"S": "ORDER-001"}}'
```

```
# Scanner la table
awslocal db scan --table-name Orders
```

```
anna@anna:~$ awslocal rds create-db-instance \
--db-instance-identifier job3-mysql \
--db-instance-class db.t3.micro \
--engine mysql \
--master-username admin \
--master-user-password SecurePass123! \
--allocated-storage 20 \
--backup-retention-period 7 \
--preferred-backup-window 03:00-04:00 \
--vpc-security-group-ids sg-a874817c437603236
{
  "DBInstance": {
    "DBInstanceIdentifier": "job3-mysql",
    "DBInstanceClass": "db.t3.micro",
    "Engine": "mysql",
    "DBInstanceStatus": "creating",
    "MasterUsername": "admin",
    "DBName": "test",
    "Endpoint": {
      "Address": "localhost.localstack.cloud",
      "Port": 4510
    },
    "AllocatedStorage": 20,
    "InstanceCreateTime": "2026-01-14T15:36:49.600149Z",
    "PreferredBackupWindow": "03:00-04:00",
    "BackupRetentionPeriod": 7,
    "DBSecurityGroups": [],
    "VpcSecurityGroups": [
      {
        "VpcSecurityGroupId": "sg-a874817c437603236",
        "Status": "active"
      }
    ],
    "DBParameterGroups": [
      {
        "DBParameterGroupName": "default.mysql8.0",
        "ParameterApplyStatus": "in-sync"
      }
    ],
    "AvailabilityZone": "us-east-1a",
    "PreferredMaintenanceWindow": "wed:06:38-wed:07:08",
    "MultiAZ": false,
    "EngineVersion": "8.0.40",
    "AutoMinorVersionUpgrade": true,
    "ReadReplicaDBInstanceIdentifiers": [],
    "LicenseModel": "general-public-license",
    "OptionGroupMemberships": [
      {
        "OptionGroupName": "default.mysql5.6",
        "Status": "in-sync"
      }
    ],
    "PubliclyAccessible": false,
    "StorageType": "gp2",
    "DbInstancePort": 4510,
    "StorageEncrypted": false,
    "DbiResourceId": "db-65D5231B",
    "CopyTagsToSnapshot": false,
    "DBInstanceArn": "arn:aws:rds:us-east-1:000000000000:db:job3-mysql",
    "IAMDatabaseAuthenticationEnabled": false
  }
}
```

localhost.localstack.cloud:4566

OverviewStatusResource BrowserState

System Status

Running

 Auto Scaling

● running

 CloudFront

● running

 EC2

● running

 Elastic Load Balancing V2

● running

 RDS

● running

 S3

● running

localhost.localstack.cloud:4566

OverviewStatusResource BrowserState

RDS Databases

RDS / Databases

Actions

Regionus-east-1

Account ID000000000000

☐	DBIdentifier	Status	Type	Engine	Region
☐	job3-mysql	available	Instance	mysql	us-east-1a

Rows per page: 1001-1 of 1

 Instance Details

RDS / Databases / rds-mysql-job3 (test)

Edit Instance

Region
us-east-1 ▼

Account ID
000000000000

[Details](#) [Query](#)

Instance Details

DBInstance Identifier
rds-mysql-job3

DBInstance Class
db.t3.micro

Engine
mysql

DBInstance Status
available

Master Username
admin

DBName
test

Endpoint

Address
localhost.localstack.cloud

Port
4510

Hosted Zone Id

localhost.localstack.cloud:4566

Overview

Status

Resource Browser

State

▼

Instance Details

RDS / Databases / job3-mysql (test)

Edit Instance

Regionus-east-1

Account ID000000000000

Details

Query

Instance Details

DBInstance Identifier

job3-mysql

DBInstance Class

db.t3.micro

Engine

mysql

DBInstance Status

available

Master Username

admin

DBName

test

Endpoint

Address

localhost.localstack.cloud

Port

4510

Hosted Zone Id

Allocated Storage

20

Preferred Backup Window

03:00-04:00

Backup Retention Period

7

DBSecurity Groups

```

anna@anna:~$ awslocal rds modify-db-instance \
  --db-instance-identifier job3-mysql \
  --backup-retention-period 7 \
  --apply-immediately

awslocal rds create-db-snapshot \
  --db-instance-identifier job3-mysql \
  --db-snapshot-identifier job3-snapshot1
{
  "DBInstance": {
    "DBInstanceIdentifier": "job3-mysql",
    "DBInstanceClass": "db.t3.micro",
    "Engine": "mysql",
    "DBInstanceStatus": "available",
    "MasterUsername": "admin",
    "DBName": "test",
    "Endpoint": {
      "Address": "localhost.localstack.cloud",
      "Port": 4510
    },
    "AllocatedStorage": 20,
    "InstanceCreateTime": "2026-01-14T15:36:49.600149Z",
    "PreferredBackupWindow": "03:00-04:00",
    "BackupRetentionPeriod": 7,
    "DBSecurityGroups": [],
    "VpcSecurityGroups": [
      {
        "VpcSecurityGroupId": "sg-a874817c437603236",
        "Status": "active"
      }
    ],
    "DBParameterGroups": [
      {
        "DBParameterGroupName": "default.mysql8.0",
        "ParameterApplyStatus": "in-sync"
      }
    ],
    "AvailabilityZone": "us-east-1a",
    "PreferredMaintenanceWindow": "wed:06:38-wed:07:08",
    "MultiAZ": false,
    "EngineVersion": "8.0.40",
    "AutoMinorVersionUpgrade": true,
    "ReadReplicaDBInstanceIdentifiers": [],
    "LicenseModel": "general-public-license",
    "OptionGroupMemberships": [
      {
        "OptionGroupName": "default.mysql5.6",
        "Status": "in-sync"
      }
    ],
    "PubliclyAccessible": false,
    "StorageType": "gp2",
    "DbInstancePort": 4510,
    "StorageEncrypted": false,
    "DbiResourceId": "db-65D5231B",
    "CopyTagsToSnapshot": false,
    "DBInstanceArn": "arn:aws:rds:us-east-1:000000000000:db:job3-mysql"
  }
}

```

```

        "VpcSecurityGroupId": "sg-a874817c437603236",
        "Status": "active"
    }
},
"DBParameterGroups": [
    {
        "DBParameterGroupName": "default.mysql8.0",
        "ParameterApplyStatus": "in-sync"
    }
],
"AvailabilityZone": "us-east-1a",
"PreferredMaintenanceWindow": "wed:06:38-wed:07:08",
"MultiAZ": false,
"EngineVersion": "8.0.40",
"AutoMinorVersionUpgrade": true,
"ReadReplicaDBInstanceIdentifiers": [],
"LicenseModel": "general-public-license",
"OptionGroupMemberships": [
    {
        "OptionGroupName": "default.mysql5.6",
        "Status": "in-sync"
    }
],
"PubliclyAccessible": false,
"StorageType": "gp2",
"DbInstancePort": 4510,
"StorageEncrypted": false,
"DbiResourceId": "db-65D5231B",
"CopyTagsToSnapshot": false,
"DBInstanceArn": "arn:aws:rds:us-east-1:000000000000:db:job3-mysql",
"IAMDatabaseAuthenticationEnabled": false,
"DeletionProtection": false,
"TagList": []
}
}
{
    "DBSnapshot": {
        "DBSnapshotIdentifier": "job3-snapshot1",
        "DBInstanceIdentifier": "job3-mysql",
        "SnapshotCreateTime": "2026-01-14T15:41:57.978269Z",
        "Engine": "mysql",
        "AllocatedStorage": 20,
        "Status": "available",
        "Port": 4510,
        "AvailabilityZone": "us-east-1a",
        "InstanceCreateTime": "2026-01-14T15:36:49.600149Z",
        "MasterUsername": "admin",
        "EngineVersion": "8.0.40",
        "LicenseModel": "general-public-license",
        "SnapshotType": "manual",
        "StorageThroughput": 0,
        "OptionGroupName": "default.mysql5.6",
        "PercentProgress": 100,
        "StorageType": "gp2",
        "Encrypted": false,
        "DBSnapshotArn": "arn:aws:rds:us-east-1:000000000000:snapshot:job3-snapshot1",
        "IAMDatabaseAuthenticationEnabled": false,
        "ProcessorFeatures": [],
        "DbiResourceId": "db-65D5231B",
        "TagList": []
    }
}

```

```
anna@anna:~$ awslocal rds describe-db-instances --db-instance-identifier job3-mysql
{
  "DBInstances": [
    {
      "DBInstanceIdentifier": "job3-mysql",
      "DBInstanceClass": "db.t3.micro",
      "Engine": "mysql",
      "DBInstanceStatus": "available",
      "MasterUsername": "admin",
      "DBName": "test",
      "Endpoint": {
        "Address": "localhost.localstack.cloud",
        "Port": 4510
      },
      "AllocatedStorage": 20,
      "InstanceCreateTime": "2026-01-14T15:36:49.600149Z",
      "PreferredBackupWindow": "03:00-04:00",
      "BackupRetentionPeriod": 7,
      "DBSecurityGroups": [],
      "VpcSecurityGroups": [
        {
          "VpcSecurityGroupId": "sg-a874817c437603236",
          "Status": "active"
        }
      ],
      "DBParameterGroups": [
        {
          "DBParameterGroupName": "default.mysql8.0",
          "ParameterApplyStatus": "in-sync"
        }
      ],
      "AvailabilityZone": "us-east-1a",
      "PreferredMaintenanceWindow": "wed:06:38-wed:07:08",
      "MultiAZ": false,
      "EngineVersion": "8.0.40",
      "AutoMinorVersionUpgrade": true,
      "ReadReplicaDBInstanceIdentifiers": [],
      "LicenseModel": "general-public-license",
      "OptionGroupMemberships": [
        {
          "OptionGroupName": "default.mysql5.6",
          "Status": "in-sync"
        }
      ],
      "PubliclyAccessible": false,
      "StorageType": "gp2",
      "DbInstancePort": 4510,
      "StorageEncrypted": false,
      "DbiResourceId": "db-65D5231B",
      "CopyTagsToSnapshot": false,
      "DBInstanceArn": "arn:aws:rds:us-east-1:000000000000:db:job3-mysql",
      "IAMDatabaseAuthenticationEnabled": false,
      "DeletionProtection": false,
      "TagList": []
    }
  ]
}
```

```

anna@anna:~$ awslocal rds create-db-snapshot \
--db-instance-identifier job3-mysql \
--db-snapshot-identifier job3-job-complete-snap

awslocal rds describe-db-snapshots \
--db-instance-identifier job3-mysql
{
  "DBSnapshot": {
    "DBSnapshotIdentifier": "job3-job-complete-snap",
    "DBInstanceIdentifier": "job3-mysql",
    "SnapshotCreateTime": "2026-01-14T16:20:29.512906Z",
    "Engine": "mysql",
    "AllocatedStorage": 20,
    "Status": "available",
    "Port": 4510,
    "AvailabilityZone": "us-east-1a",
    "InstanceCreateTime": "2026-01-14T15:36:49.600149Z",
    "MasterUsername": "admin",
    "EngineVersion": "8.0.40",
    "LicenseModel": "general-public-license",
    "SnapshotType": "manual",
    "StorageThroughput": 0,
    "OptionGroupName": "default.mysql5.6",
    "PercentProgress": 100,
    "StorageType": "gp2",
    "Encrypted": false,
    "DBSnapshotArn": "arn:aws:rds:us-east-1:000000000000:snapshot:job3-job-complete-snap",
    "IAMDatabaseAuthenticationEnabled": false,
    "ProcessorFeatures": [],
    "DbiResourceId": "db-65D5231B",
    "TagList": [],
    "SnapshotTarget": "region",
    "OriginalSnapshotCreateTime": "2026-01-14T16:20:29.512921Z"
  }
}
{
  "DBSnapshots": [
    {
      "DBSnapshotIdentifier": "job3-snapshot1",
      "DBInstanceIdentifier": "job3-mysql",
      "SnapshotCreateTime": "2026-01-14T15:41:57.978269Z",
      "Engine": "mysql",
      "AllocatedStorage": 20,
      "Status": "available",
      "Port": 4510,
      "AvailabilityZone": "us-east-1a",
      "InstanceCreateTime": "2026-01-14T15:36:49.600149Z",
      "MasterUsername": "admin",
      "EngineVersion": "8.0.40",
      "LicenseModel": "general-public-license",
      "SnapshotType": "manual",
      "StorageThroughput": 0,
      "OptionGroupName": "default.mysql5.6",
      "PercentProgress": 100,
      "StorageType": "gp2",
      "Encrypted": false,
      "DBSnapshotArn": "arn:aws:rds:us-east-1:000000000000:snapshot:job3-snapshot1",
      "IAMDatabaseAuthenticationEnabled": false,
      "ProcessorFeatures": []
    }
  ]
}

```

Instance Details

[RDS](#) / [Databases](#) / [job3-mysql \(test\)](#)

Edit Instance

Region
us-east-1

Account ID
000000000000

Details

Query

Instance Details

DBInstance Identifier
job3-mysql

DBInstance Class
db.t3.micro

Engine
mysql

DBInstance Status
available

Master Username
admin

DBName
test

Endpoint

Address
localhost.localstack.cloud

Port
4510

Hosted Zone Id

Allocated Storage
20

Preferred Backup Window
03:00-04:00

Backup Retention Period
7

DBSecurity Groups

JOB4 : API Restful Serverless

Lambda: service de calcul du code

API Gateway: permet de créer, de publier, de gérer, de surveiller et de sécuriser les API REST, HTTP et Websocket à n'importe quelle échelle.

CloudWatch Logs: permet de collecter et de stocker les journaux des ressources, applications et services en temps réel.

Créer la fonction Lambda (handler.py)

```
```python
import json

def lambda_handler(event, context):
 return {
```

```

 'statusCode': 200,
 'headers': {'Content-Type': 'application/json'},
 'body': json.dumps({
 'message': 'Hello from LocalStack Lambda!',
 'path': event.get('path', '/')
 })
 }
}

```

### Zipper et déployer

# Zipper la fonction

```
zip function.zip handler.py
```

# Créer la fonction Lambda

```

awslocal lambda create-function \
 --function-name MyAPIFunction \
 --runtime python3.11 \
 --role arn:aws:iam::000000000000:role/lambda-role \
 --handler handler.lambda_handler \
 --zip-file fileb://function.zip

```

# Tester la fonction

```

awslocal lambda invoke \
 --function-name MyAPIFunction \
 --payload '{"path": "/test"}' \
 response.json

```

```
cat response.json
```

```
```
```

Créer l'API Gateway

Créer l'API REST

```

awslocal apigateway create-rest-api \
    --name MyAPI \
    --description "API de test LocalStack"

```

Obtenir l'API ID

```

API_ID=$(awslocal apigateway get-rest-apis \
    --query 'items[0].id' --output text)

```

Obtenir le root resource ID

```

ROOT_ID=$(awslocal apigateway get-resources \
    --rest-api-id $API_ID \
    --query 'items[0].id' --output text)

```

Créer une méthode GET

```

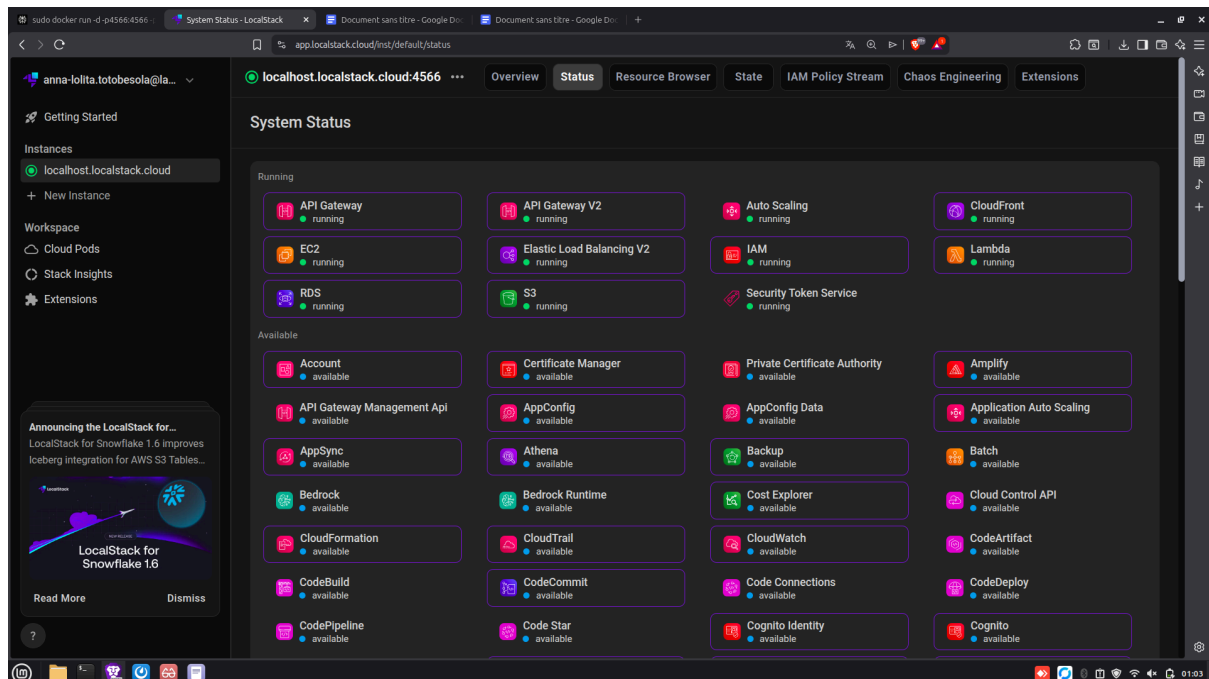
awslocal apigateway put-method \
    --rest-api-id $API_ID \
    --resource-id $ROOT_ID \
    --http-method GET \
    --authorization-type NONE

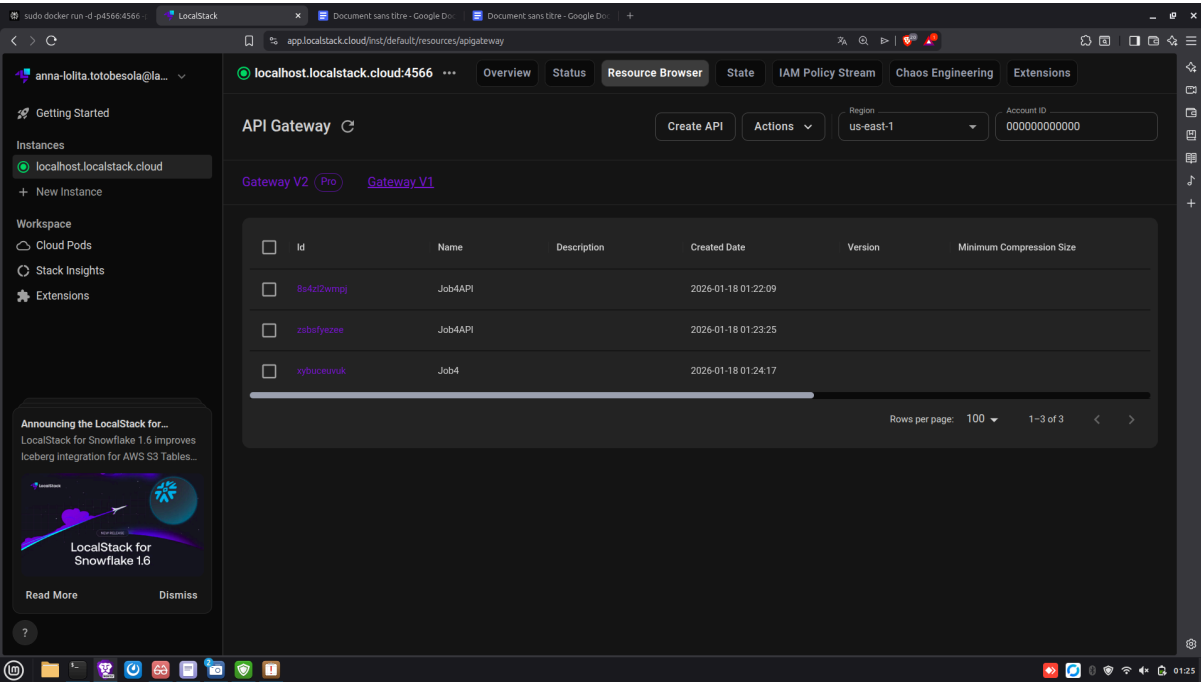
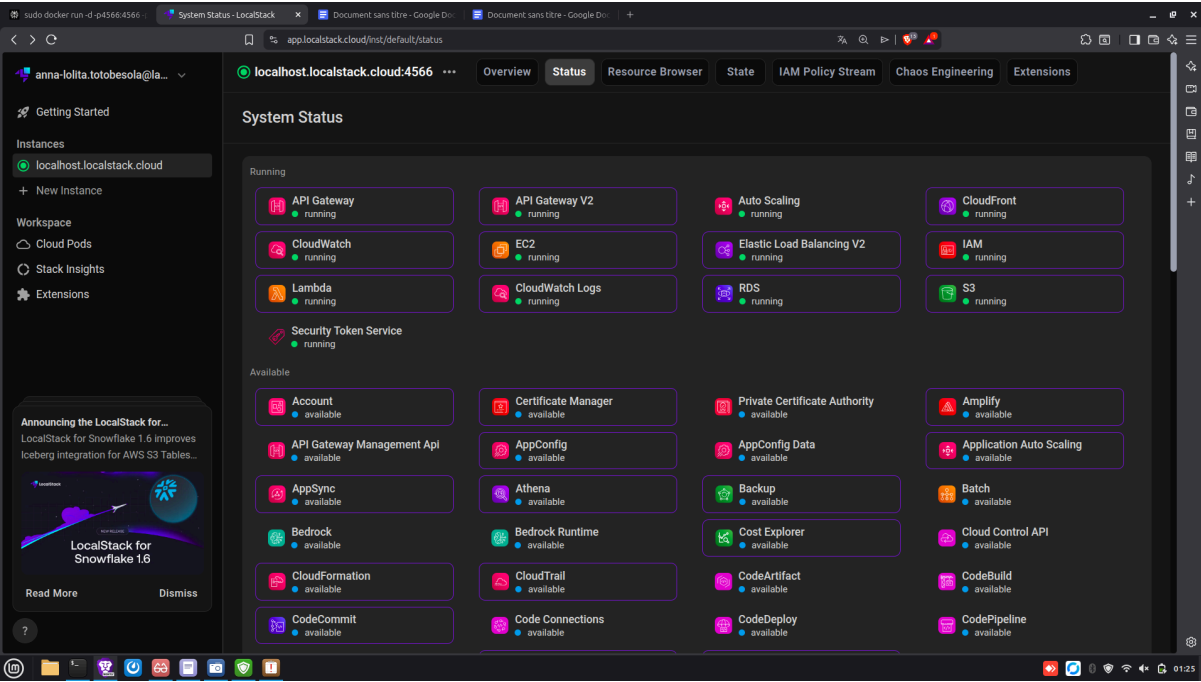
```

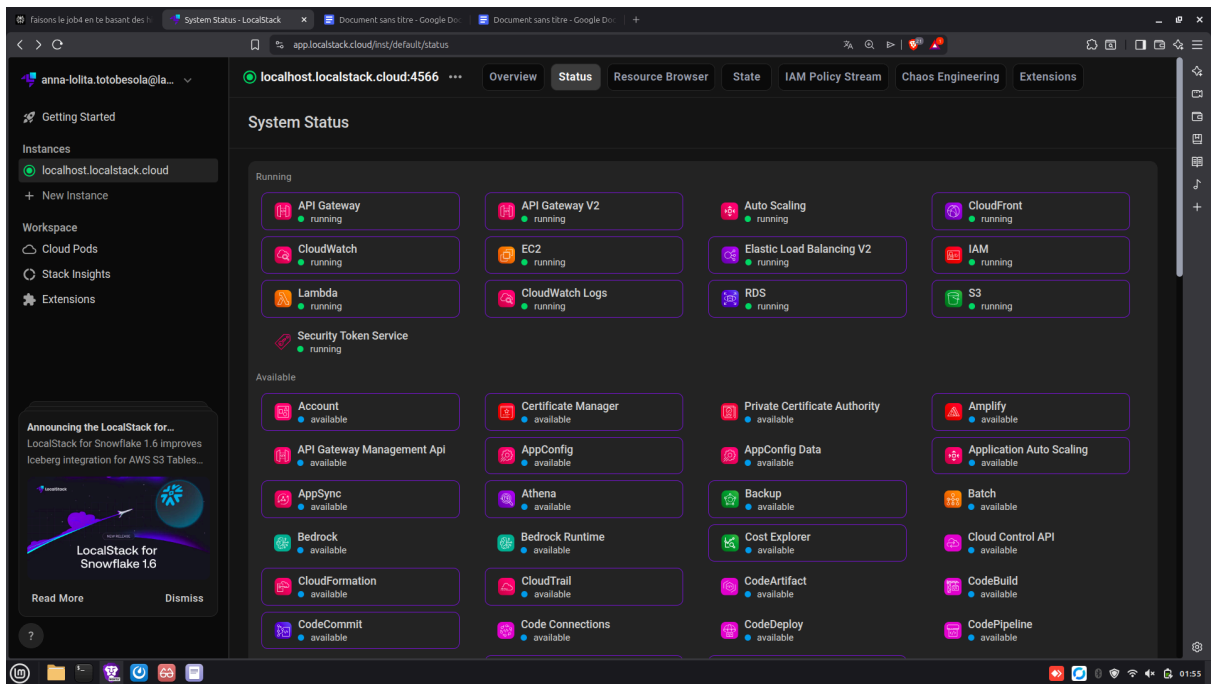
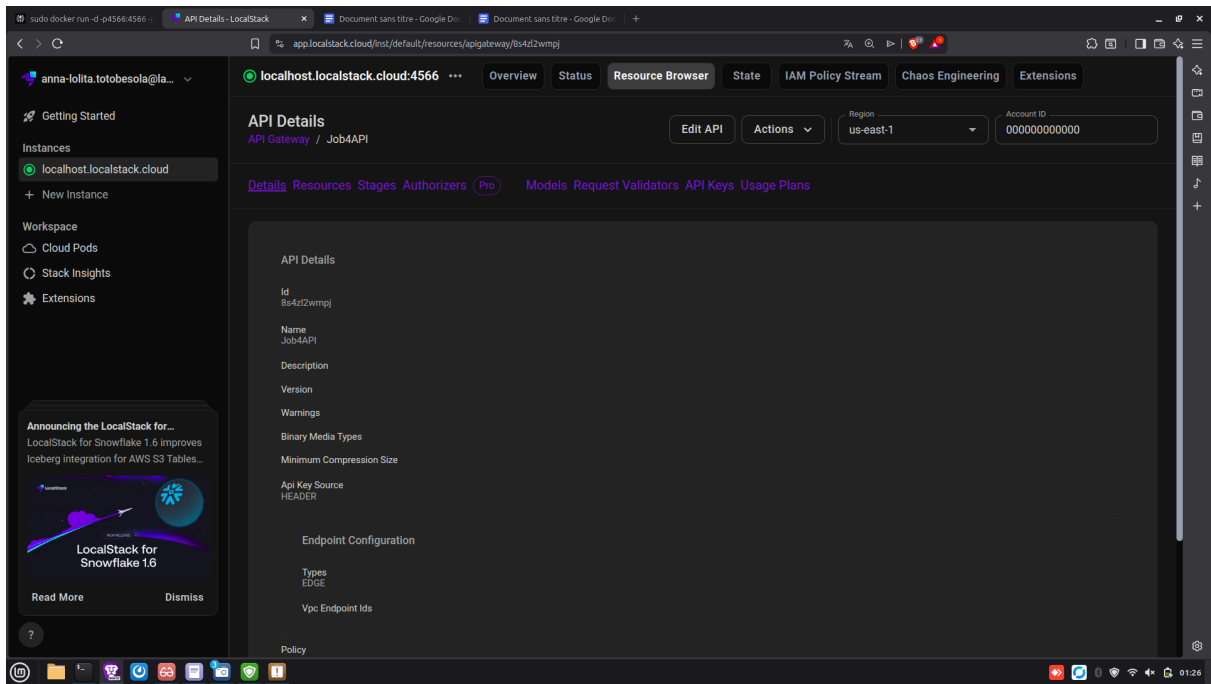
```
# Intégrer avec Lambda
awslocal apigateway put-integration \
  --rest-api-id $API_ID \
  --resource-id $ROOT_ID \
  --http-method GET \
  --type AWS_PROXY \
  --integration-http-method POST \
  --uri
arn:aws:apigateway:us-east-1:lambda:path/2015-03-31/functions/arn:aws:lambda:us-east-1:000000000000:function:MyAPIFunction/invocations

# Déployer l'API
awslocal apigateway create-deployment \
  --rest-api-id $API_ID \
  --stage-name prod

# Tester l'API
curl http://localhost:4566/restapis/$API_ID/prod/_user_request/
```







LocalStack

Document sans titre - Google Doc

Document sans titre - Google Doc

app.localstack.cloud/inst/default/resources/cloudwatch

anna-lolita.totobesola@la...

Getting Started

Instances

localhost.localstack.cloud

New Instance

Workspace

Cloud Pods

Stack Insights

Extensions

Announcing the LocalStack for...

LocalStack for Snowflake 1.6 improves Iceberg integration for AWS S3 Tables...

Read More

Dismiss

localhost.localstack.cloud:4566

Overview

Status

Resource Browser

State

IAM Policy Stream

Chaos Engineering

Extensions

CloudWatch Metrics

Put Metric

Regionus-east-1

Account ID000000000000

Metrics Alarms

| Metric Name | Namespace | Dimensions |
|-------------|------------|--|
| Errors | AWS/Lambda | [{"Name": "FunctionName", "Value": ...}] |
| Invocations | AWS/Lambda | [{"Name": "FunctionName", "Value": ...}] |

Rows per page: 1001-2 of 2

LocalStack

Document sans titre - Google Doc

Document sans titre - Google Doc

app.localstack.cloud/inst/default/resources/cloudwatch/groups

anna-lolita.totobesola@la...

Getting Started

Instances

localhost.localstack.cloud

New Instance

Workspace

Cloud Pods

Stack Insights

Extensions

Announcing the LocalStack for...

LocalStack for Snowflake 1.6 improves Iceberg integration for AWS S3 Tables...

Read More

Dismiss

localhost.localstack.cloud:4566

Overview

Status

Resource Browser

State

IAM Policy Stream

Chaos Engineering

Extensions

CloudWatch Log Groups

Create Log Group

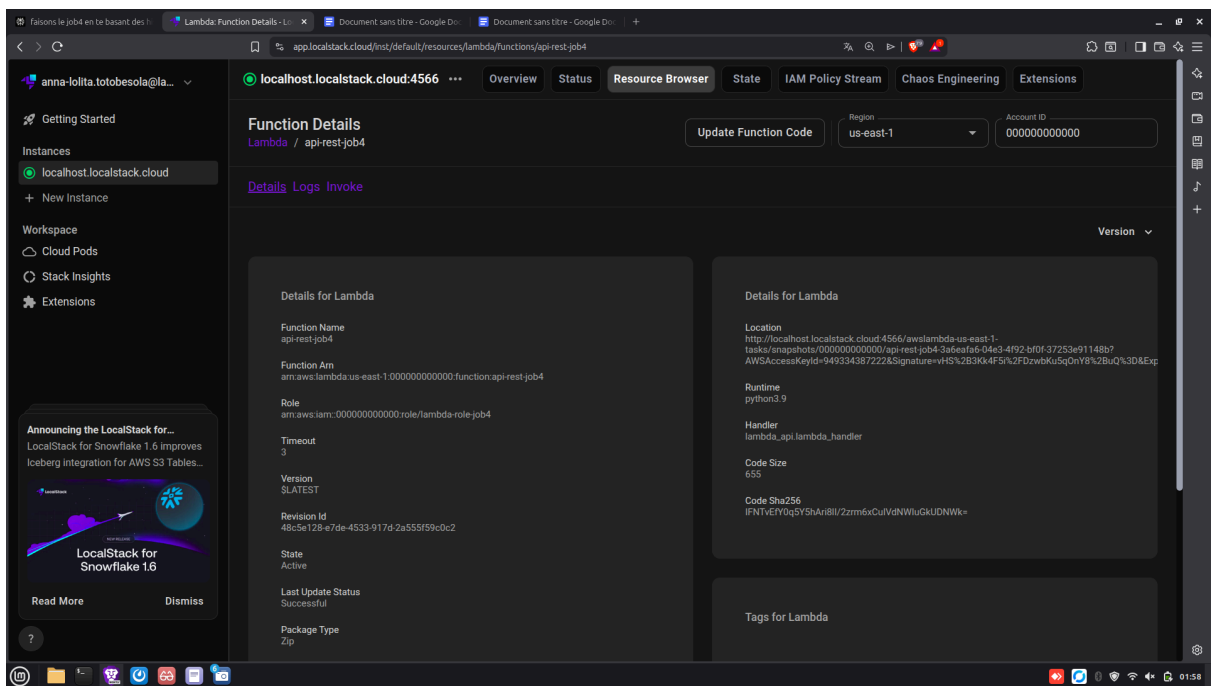
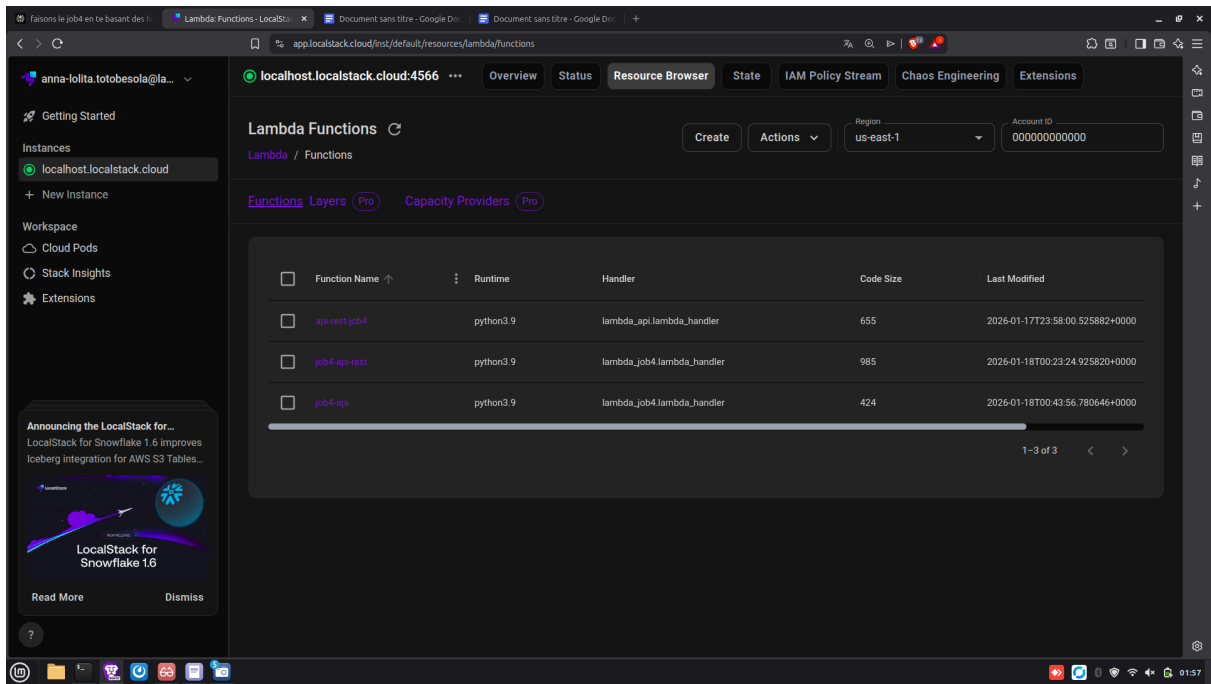
Actions

Regionus-east-1

Account ID000000000000

| Log Group Name | Creation Time | Retention In Days | Metric Filter Count | Arn |
|----------------------|---------------------|-------------------|---------------------|---|
| /aws/lambda/job4-api | 2026-01-18 01:43:58 | | 0 | arn:aws:logs:us-east-1:0000000000000000 |

Rows per page: 1001-1 of 1



Job5 : Surveillances et Alertes

CloudWatch: surveille et monitor les ressources et les applications

SNS: service de notifications en temps réel

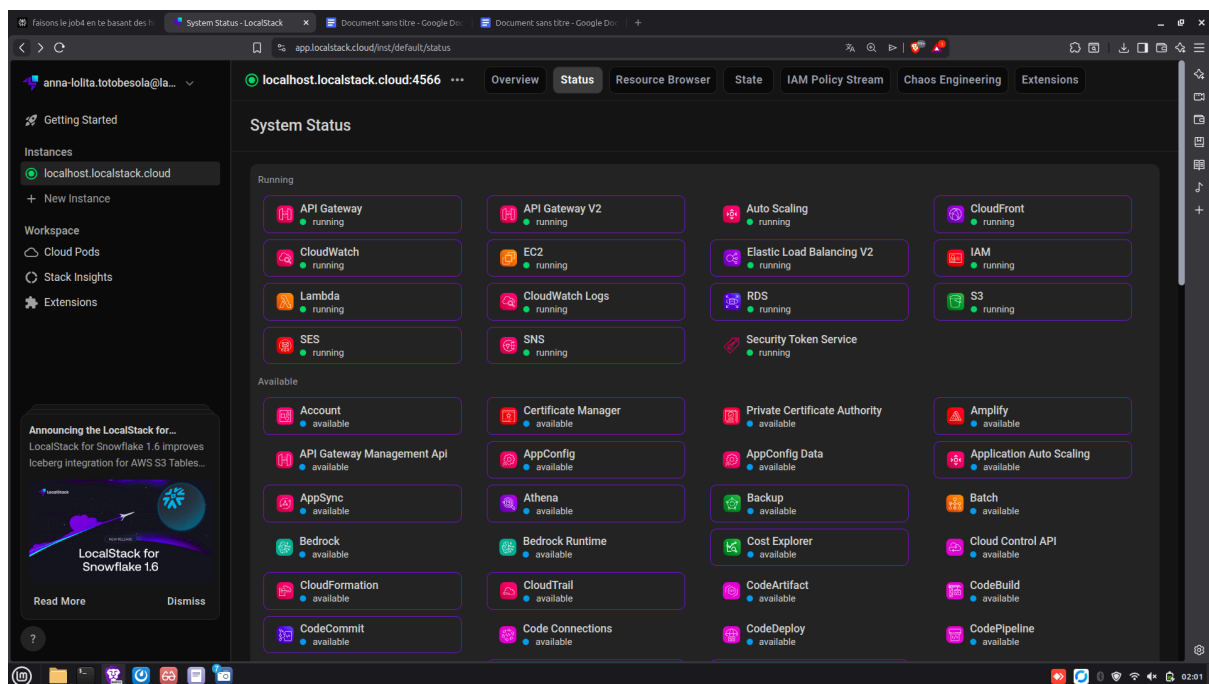
```
# Créer un topic SNS
awslocal sns create-topic --name CloudWatch-Alerts

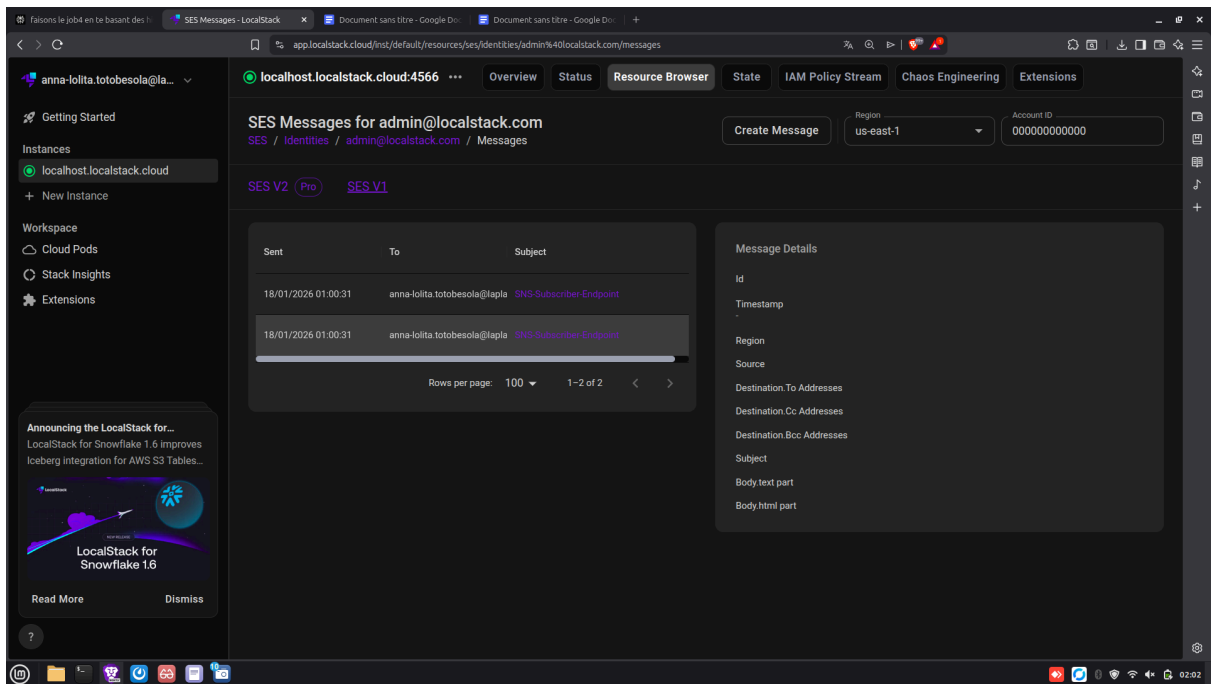
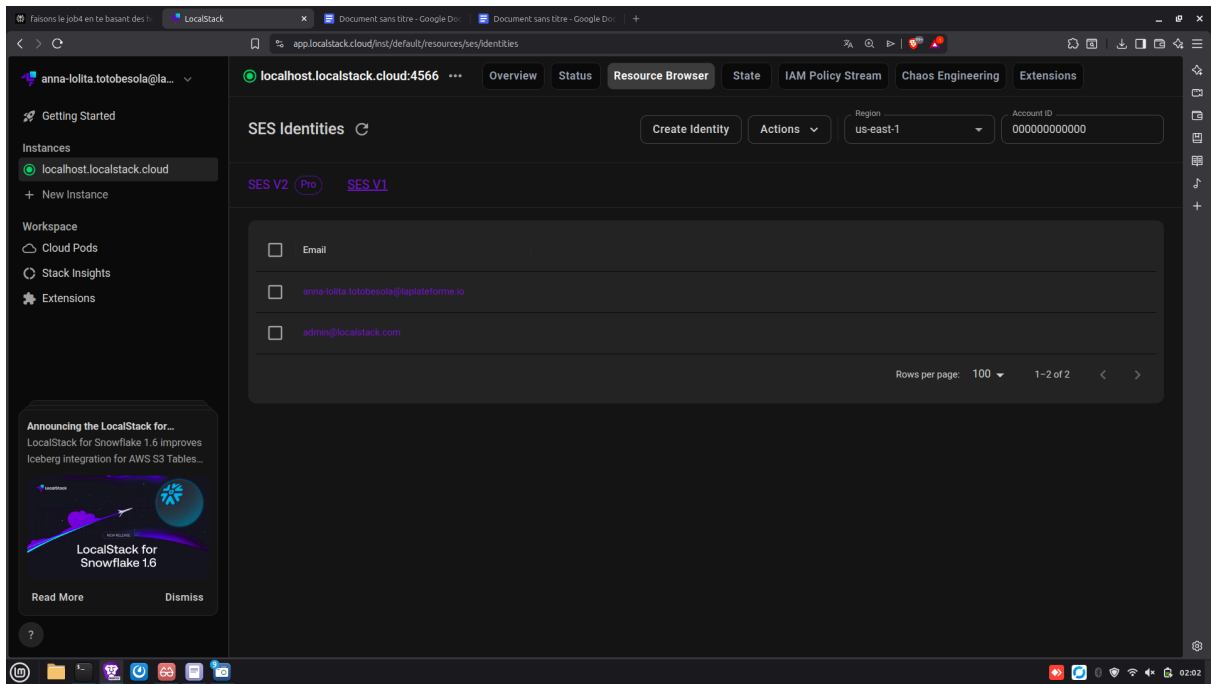
# S'abonner au topic (email simulé)
awslocal sns subscribe \
  --topic-arn arn:aws:sns:us-east-1:000000000000:CloudWatch-Alerts \
  --protocol email \
  --notification-endpoint test@example.com

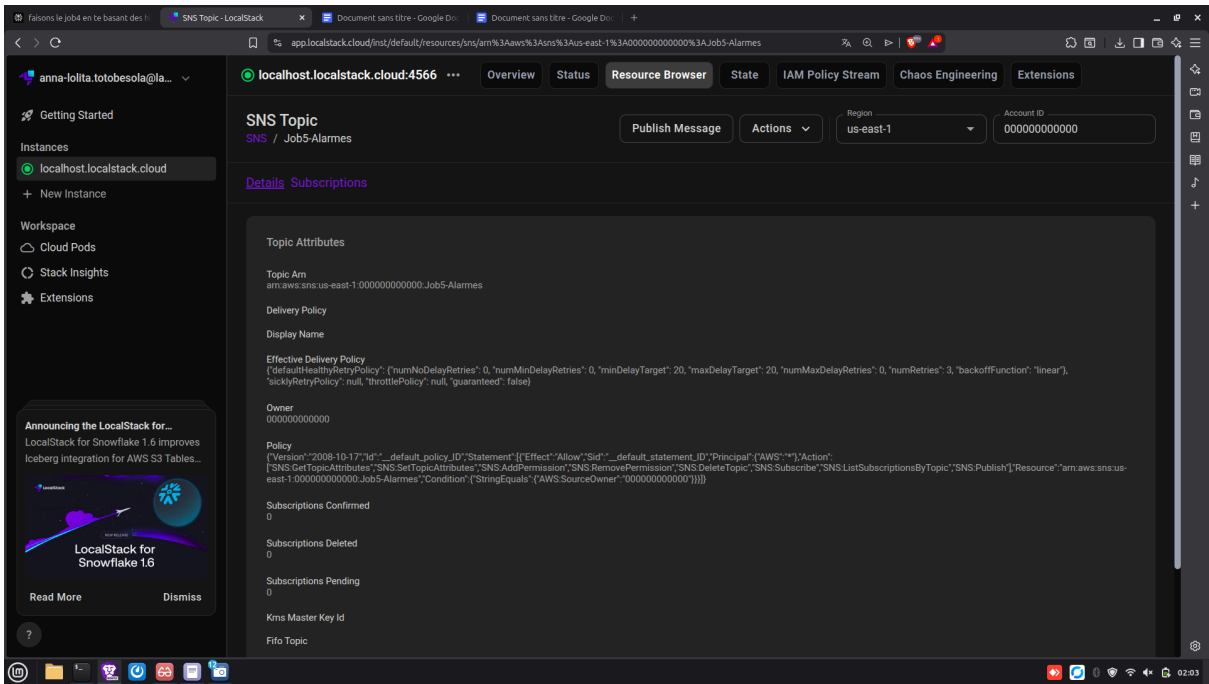
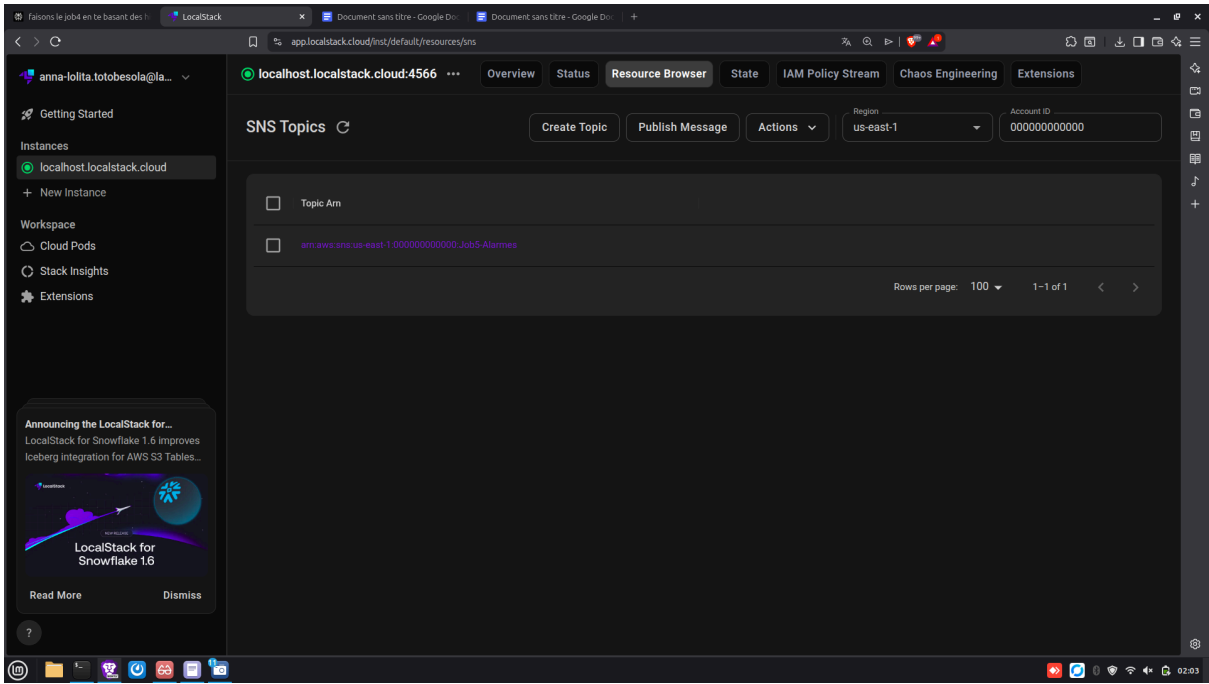
# Publier une métrique custom
awslocal cloudwatch put-metric-data \
  --namespace "CustomApp" \
  --metric-name "CPUUtilization" \
  --value 85.0

# Créer une alarme
awslocal cloudwatch put-metric-alarm \
  --alarm-name high-cpu \
  --comparison-operator GreaterThanThreshold \
  --evaluation-periods 1 \
  --metric-name CPUUtilization \
  --namespace CustomApp \
  --period 60 \
  --statistic Average \
  --threshold 70.0 \
  --alarm-actions arn:aws:sns:us-east-1:000000000000:CloudWatch-Alerts

# Lister les alarmes
awslocal cloudwatch describe-alarms
```







```
anna@anna: ~/telechargements
anna@anna:~$ http://localhost:4566/restapis/8mmintypwf/prod/_user_request_/
bash: http://localhost:4566/restapis/8mmintypwf/prod/_user_request_/: Aucun fichier ou dossier de ce nom
anna@anna:~$ awslocal cloudwatch describe-alarms --alarm-name-prefix Job5
{
  "MetricAlarms": [
    {
      "AlarmName": "Job5-Lambda-Slow",
      "AlarmArn": "arn:aws:cloudwatch:us-east-1:000000000000:alarm:Job5-Lambda-Slow",
      "AlarmConfigurationUpdatedTimestamp": 1768698030.927657,
      "ActionsEnabled": true,
      "OKActions": [
        "arn:aws:sns:us-east-1:000000000000:Job5-Alarmes"
      ],
      "AlarmActions": [
        "arn:aws:sns:us-east-1:000000000000:Job5-Alarmes"
      ],
      "InsufficientDataActions": [],
      "StateValue": "OK",
      "StateReason": "Threshold Crossed: no datapoints were received for 2 periods and 2 missing datapoints were treated as [NonBreaching].",
      "StateReasonData": "{\n  \"version\": \"1.0\", \"queryDate\": \"2026-01-18T01:00:30+0000\", \"period\": 300, \"recentDatapoints\": [], \"threshold\": 500.0, \"statistic\": \"Average\", \"startDate\": \"2026-01-18T01:00:30+0000\"}",
      "StateUpdatedTimestamp": 1768698030.968411,
      "MetricName": "Duration",
      "Namespace": "AWS/Lambda",
      "Statistic": "Average",
      "Dimensions": [
        {
          "Name": "FunctionName",
          "Value": "job4-api"
        }
      ],
      "Period": 300,
      "EvaluationPeriods": 2,
      "Threshold": 500.0,
      "ComparisonOperator": "GreaterThanThreshold",
      "TreatMissingData": "FillByPreviousValue"
    }
  ]
}
```

JOB6: Pipeline de données

AWS GLue élimine la gestion de l'infra en fournissant des pipelines de données sans serveur dotés

```
#!/bin/bash
```

```
echo "Job 6 : AWS Glue ETL Pipeline S3→Crawler→ETL"
```

Étape 1 : S3 Buckets

```
echo "Étape 1 : Buckets S3"
```

```
awslocal s3 mb s3://glue-raw-data
```

```
awslocal s3 mb s3://glue-clean-data
```

Données brutes

```
cat > sales-raw.csv << 'EOF'
```

```
date,user_id,product,amount,status
```

```
2026-01-01,anna123,api_calls,1500.50,OK
```

```
2026-01-01,user456,lambda,999.99,ERROR
```

```
2026-01-02,anna123,job5_alerts,250.00,OK
```

```
2026-01-02,user789,glue_etl,0.00,CANCELLED
```

```
EOF
```

```
awslocal s3 cp sales-raw.csv s3://glue-raw-data/sales-raw.csv
```

```
echo "Brutes : s3://glue-raw-data/sales-raw.csv"
```

Étape 2 : IAM Role Glue

```
echo "Étape 2 : IAM Role"
```

```
ROLE_ARN=$(awslocal iam create-role \
```

```
--role-name glue-job6-role \
```

```
--assume-role-policy-document '{
```

```
"Version": "2012-10-17",
"Statement": [{"Effect": "Allow", "Principal": {"Service": "glue.amazonaws.com"}, "Action":
"sts:AssumeRole"}]
}' | jq -r '.Role.Arn')
```

```
awslocal iam attach-role-policy --role-name glue-job6-role \
--policy-arn arn:aws:iam::aws:policy/service-role/AWSGlueServiceRole
```

Étape 3 : Glue Database

```
echo " Étape 3 : Glue Database"
```

```
awslocal glue create-database --database-input '{"Name": "job6_db"}'
```

Étape 4 : Glue Crawler (détecte schéma)

```
echo " Étape 4 : Crawler Schéma"
```

```
CRAWLER_NAME="job6-sales-crawler"
```

```
awslocal glue create-crawler \
```

```
--name $CRAWLER_NAME \
```

```
--role $ROLE_ARN \
```

```
--targets '{"S3Targets": [{"Path": "s3://glue-raw-data/"}]}'
```

```
awslocal glue start-crawler --name $CRAWLER_NAME
```

```
sleep 15 # Crawler time
```

Vérif table créée

```
echo "Tables détectées :"
```

```
awslocal glue get-tables --database-name job6_db
```

Étape 5 : Glue ETL Job (transformation)

```
echo "Étape 5 : ETL Job"
```

```
cat > glue-etl.py << 'EOF'
```

```
import sys
```

```
from awsglue.transforms import *
```

```
from awsglue.utils import getResolvedOptions
```

```
from pyspark.context import SparkContext
```

```
from awsglue.context import GlueContext
```

```
from awsglue.job import Job
```

```
args = getResolvedOptions(sys.argv, ['JOB_NAME'])
```

```
sc = SparkContext()
```

```
glueContext = GlueContext(sc)
```

```
spark = glueContext.spark_session
```

```
job = Job(glueContext)
```

```
job.init(args['JOB_NAME'], args)
```

Lire catalogue (crawler)

```
raw_df = glueContext.create_dynamic_frame.from_catalog(
```

```
    database = "job6_db",
```

```
    table_name = "sales_raw")
```

Transformer : Filtrer OK + amount > 0

```

clean_df = Filter.apply(
    frame = raw_df,
    f = lambda x: x["amount"] > 0 and x["status"] == "OK")

# Ajouter colonne revenue_net
from pyspark.sql.functions import col, round
spark_df = clean_df.toDF()
transformed_df = spark_df.withColumn("revenue_net",
    round(col("amount") * 0.9, 2)).toDF()

glue_df = DynamicFrame.fromDF(transformed_df, glueContext, "transformed")

# Sauvegarder Parquet nettoyé
glueContext.write_dynamic_frame.from_options(
    frame = glue_df,
    connection_type = "s3",
    connection_options = {"path": "s3://glue-clean-data/sales-cleaned/"},
    format = "parquet")

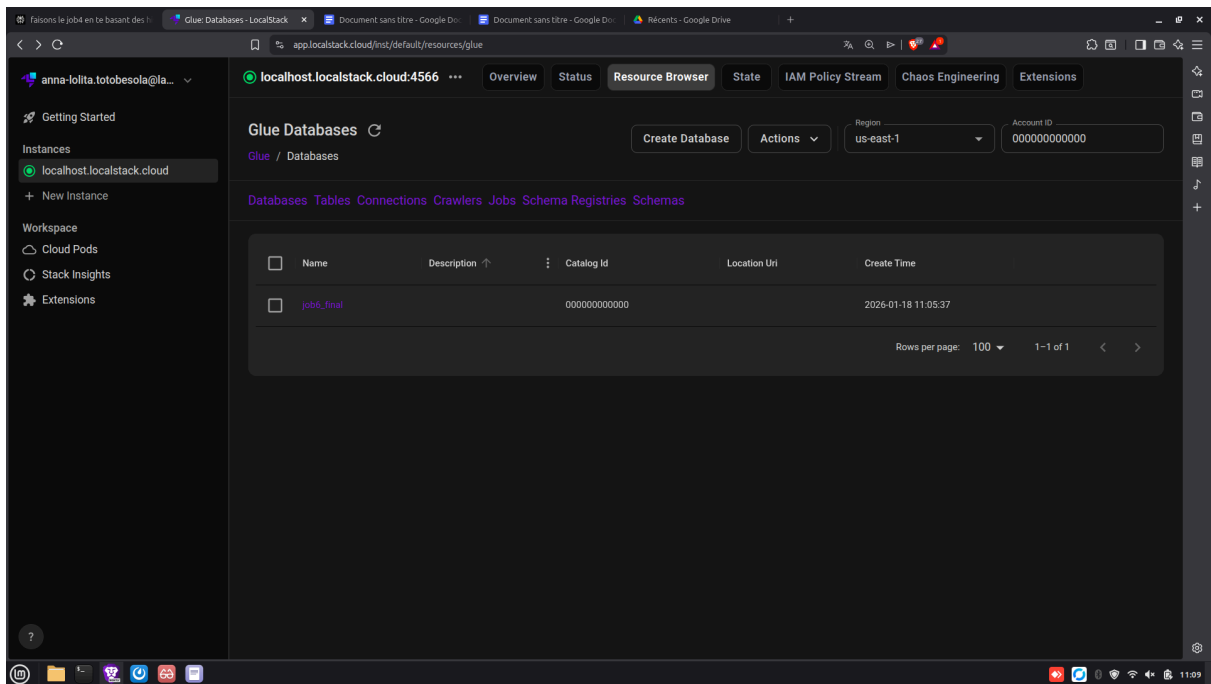
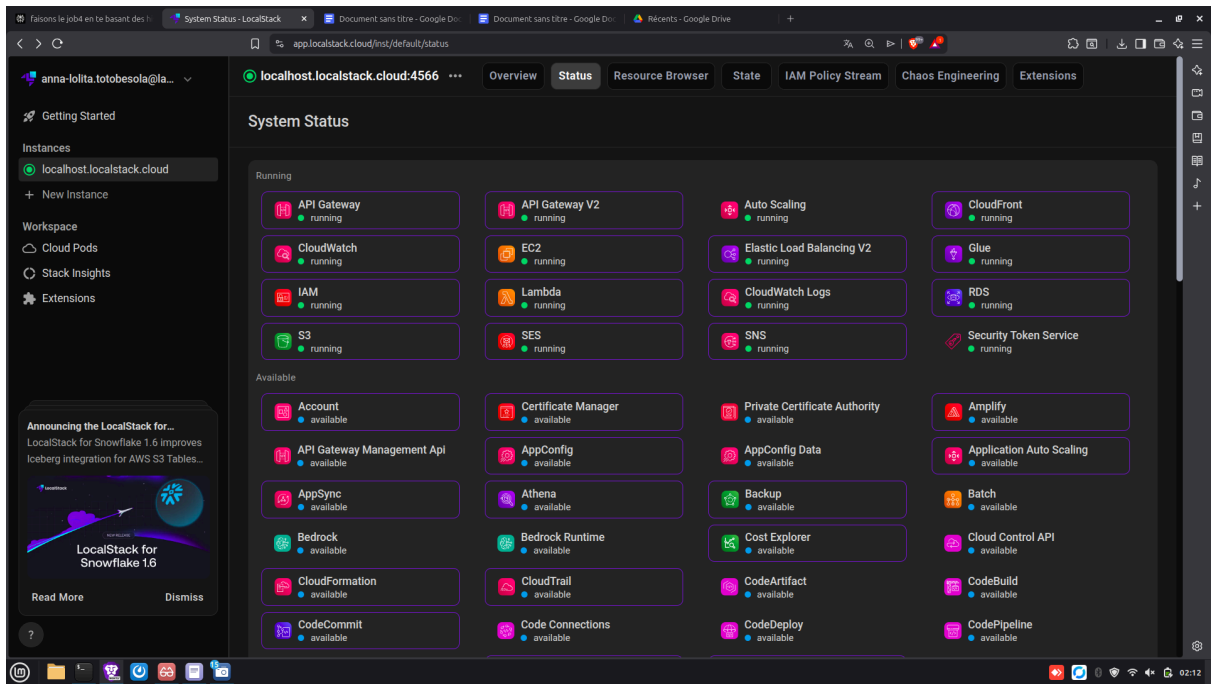
job.commit()
EOF

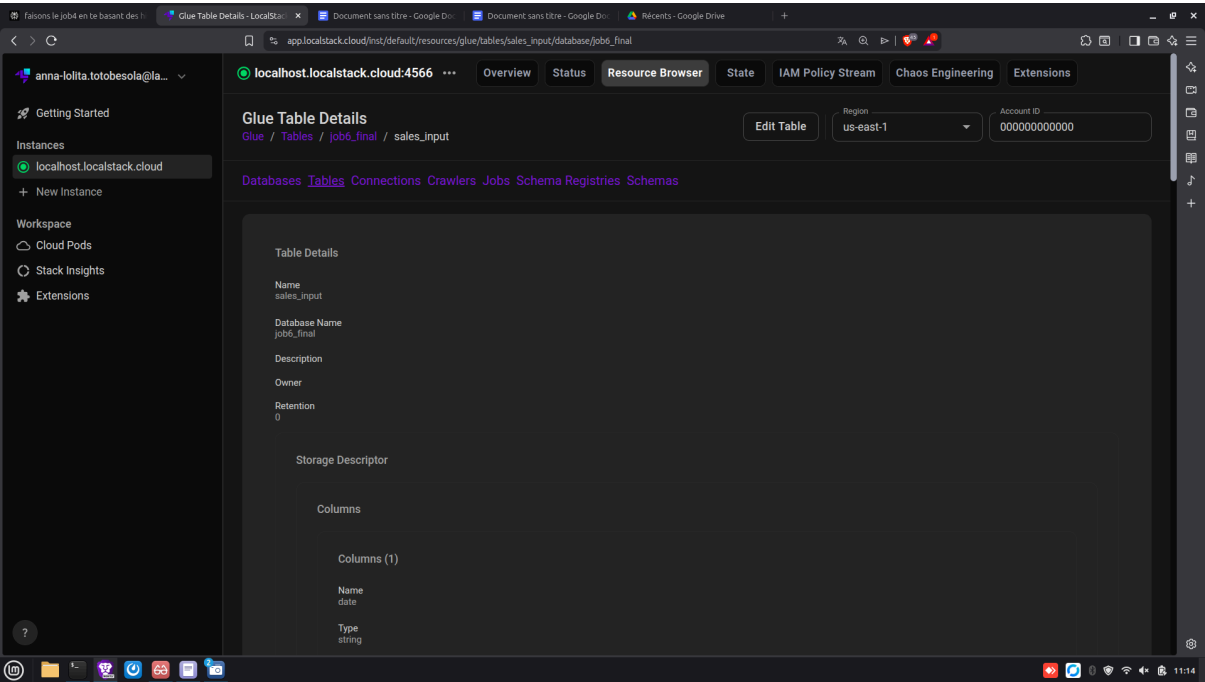
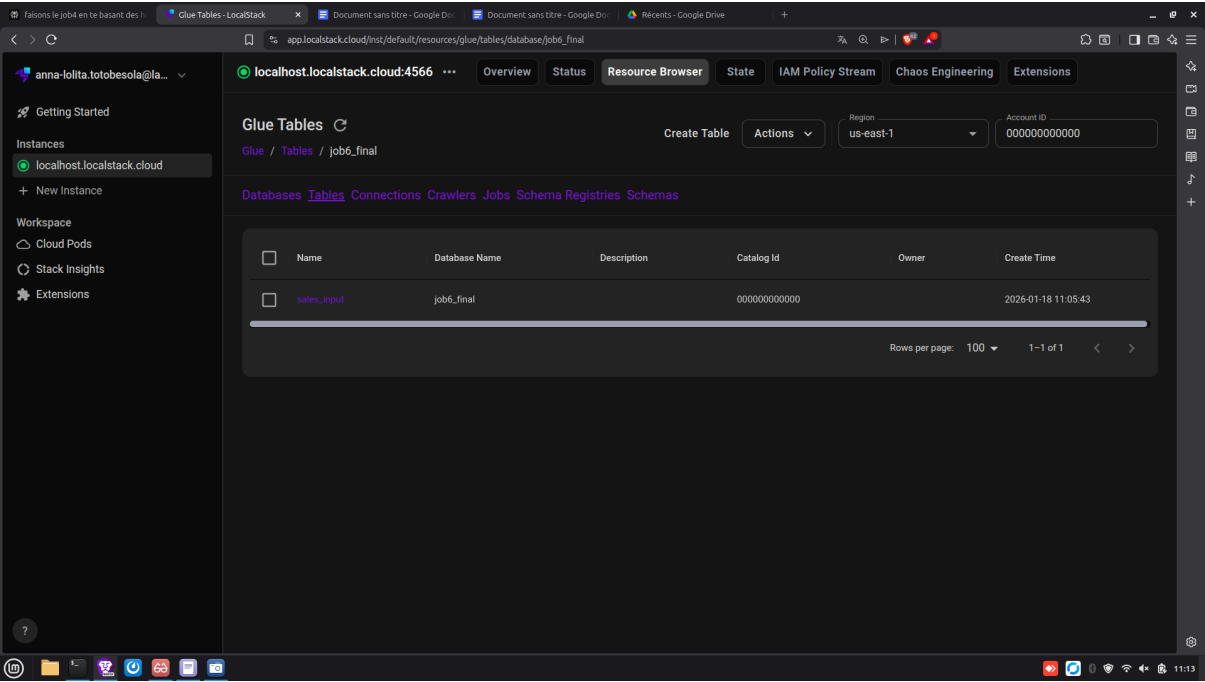
# Upload script + créer job
awslocal s3 cp glue-etl.py s3://glue-raw-data/glue-etl.py
awslocal glue create-job \
    --name job6-sales-etl \
    --role $ROLE_ARN \
    --command '{"Name": "glueetl", "ScriptLocation": "s3://glue-raw-data/glue-etl.py"}'

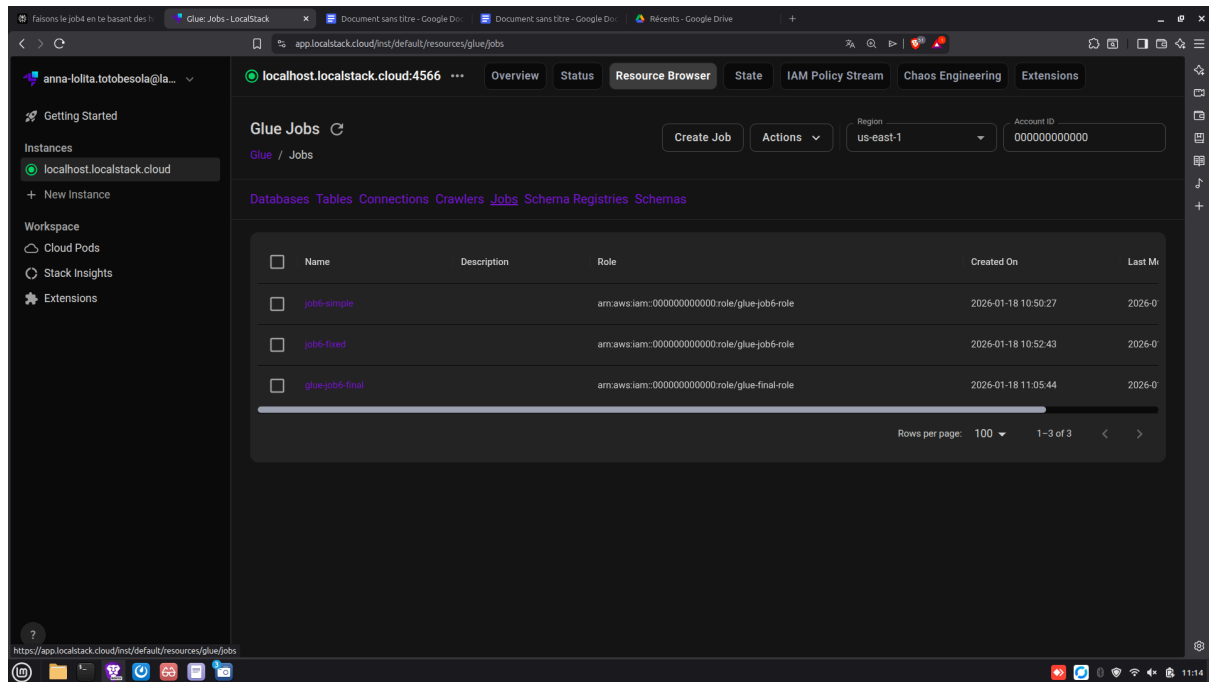
# Lancer ETL
awslocal glue start-job-run --job-name job6-sales-etl

echo "Job 6 TERMINÉ ! Résultats :"
```

- echo "1. Brutes : s3://glue-raw-data/sales-raw.csv (4 lignes)"
- echo "2. Table Glue : job6_db.sales_raw (crawler)"
- echo "3. Nettoyées : s3://glue-clean-data/sales-cleaned/ (2 lignes OK)"







```
# Créer les buckets pour le data lake
awslocal s3 mb s3://datalake-raw
awslocal s3 mb s3://datalake-processed
```

```
# Uploader des données brutes
cat > sales.csv << EOF
order_id,product,quantity,price
1,Laptop,1,999.99
2,Mouse,2,29.99
3,Keyboard,1,79.99
EOF
```

```
awslocal s3 cp sales.csv s3://datalake-raw/sales/
```

```
# Script ETL Python (transform.py)
cat > transform.py << 'EOF'
import boto3
import csv
import json
```

```
s3 = boto3.client('s3', endpoint_url='http://localhost:4566')
```

```
# Lire depuis S3
obj = s3.get_object(Bucket='datalake-raw', Key='sales/sales.csv')
data = obj['_Body'].read().decode('utf-8')
```

```
# Transformer
reader = csv.DictReader(data.splitlines())
processed = []
for row in reader:
```

```

        row['total'] = float(row['quantity']) * float(row['price'])
    processed.append(row)

# Écrire dans S3 processed
s3.put_object(
    Bucket='datalake-processed',
    Key='sales/sales_processed.json',
    Body=json.dumps(processed, indent=2)
)
print("ETL completed!")
EOF

# Exécuter l'ETL
pip install boto3
python transform.py

# Vérifier le résultat
awslocal s3 cp s3://datalake-processed/sales/sales_processed.json -

```

JOB7 : Analyse de données

Athena: service de requêtes interactif permettant d'analyser de grands ensembles de données directement dans AWS S3 en utilisant le SQL standard

QuickSight: service de BI cloud-native qui permet aux entreprises de visualiser, d'analyser et de partager des données à grande échelle.

```

```python
Script d'analyse Python (analyze.py)
import boto3
import json
import pandas as pd

s3 = boto3.client('s3', endpoint_url='http://localhost:4566')

Lire les données depuis S3
obj = s3.get_object(Bucket='datalake-processed', Key='sales/sales_processed.json')
data = json.loads(obj['Body'].read().decode('utf-8'))

Convertir en DataFrame pandas
df = pd.DataFrame(data)

Analyses SQL-like
print("=== Total Revenue ===")
print(f"${df['total'].sum():.2f}")

print("\n=== Revenue by Product ===")
print(df.groupby('product')['total'].sum().sort_values(ascending=False))

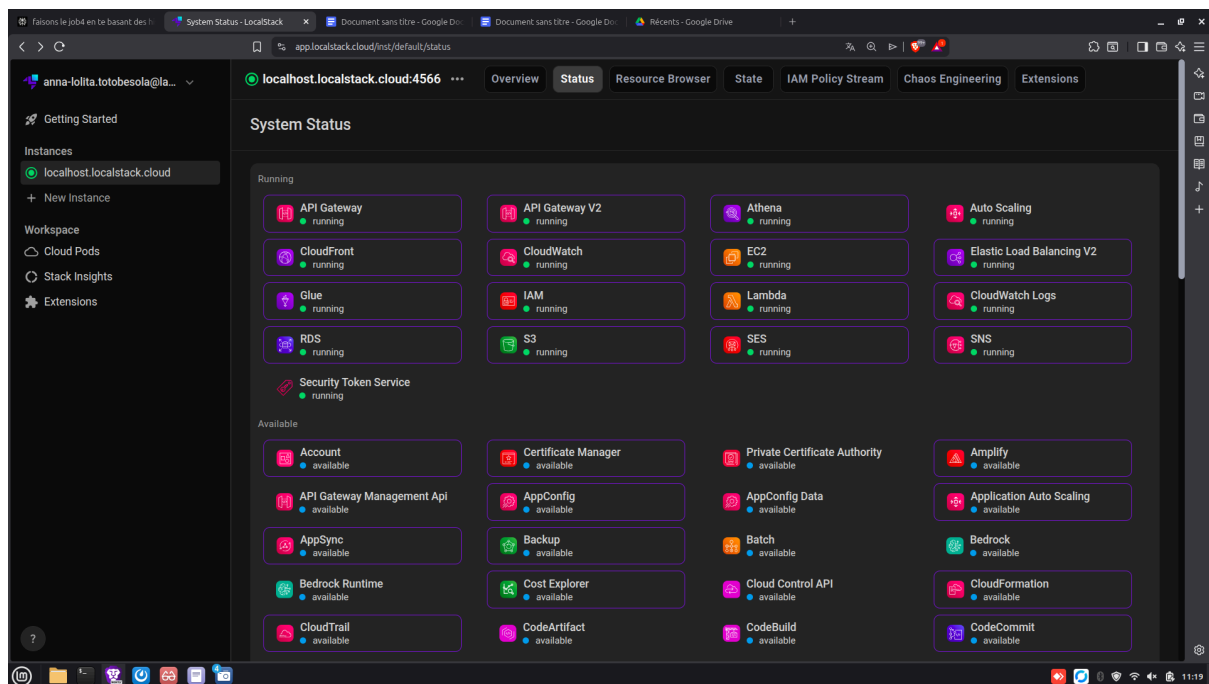
```

```
print("\n=== Top 5 Orders ===")
print(df.nlargest(5, 'total')[['order_id', 'product', 'total']])
```

```
Générer un rapport HTML
html_report = df.to_html()
s3.put_object(
 Bucket='datalake-processed',
 Key='reports/sales_report.html',
 Body=html_report,
 ContentType='text/html'
)
print("\nReport saved to S3!")
'''
```

```
'''bash
Installer pandas
pip install pandas

Exécuter l'analyse
python analyze.py
```



faisons le job4 en le basant des i Athena Databases - LocalStack Document sans titre - Google Doi Document sans titre - Google Doi Récents - Google Drive

app.localstack.cloud/inst/default/resources/athena/databases

anna-lolita.totobesola@la... localhost.localstack.cloud:4566 Overview Status Resource Browser State IAM Policy Stream Chaos Engineering Extensions

### Athena Databases

Region: us-east-1 Account ID: 000000000000

Databases Catalogs SQL

Name	Description
default	
job6_final	

Rows per page: 100 1-2 of 2

faisons le job4 en le basant des i Database Tables - LocalStack Document sans titre - Google Doi Document sans titre - Google Doi Récents - Google Drive

app.localstack.cloud/inst/default/resources/athena/databases/job6\_final

anna-lolita.totobesola@la... localhost.localstack.cloud:4566 Overview Status Resource Browser State IAM Policy Stream Chaos Engineering Extensions

### Database Tables

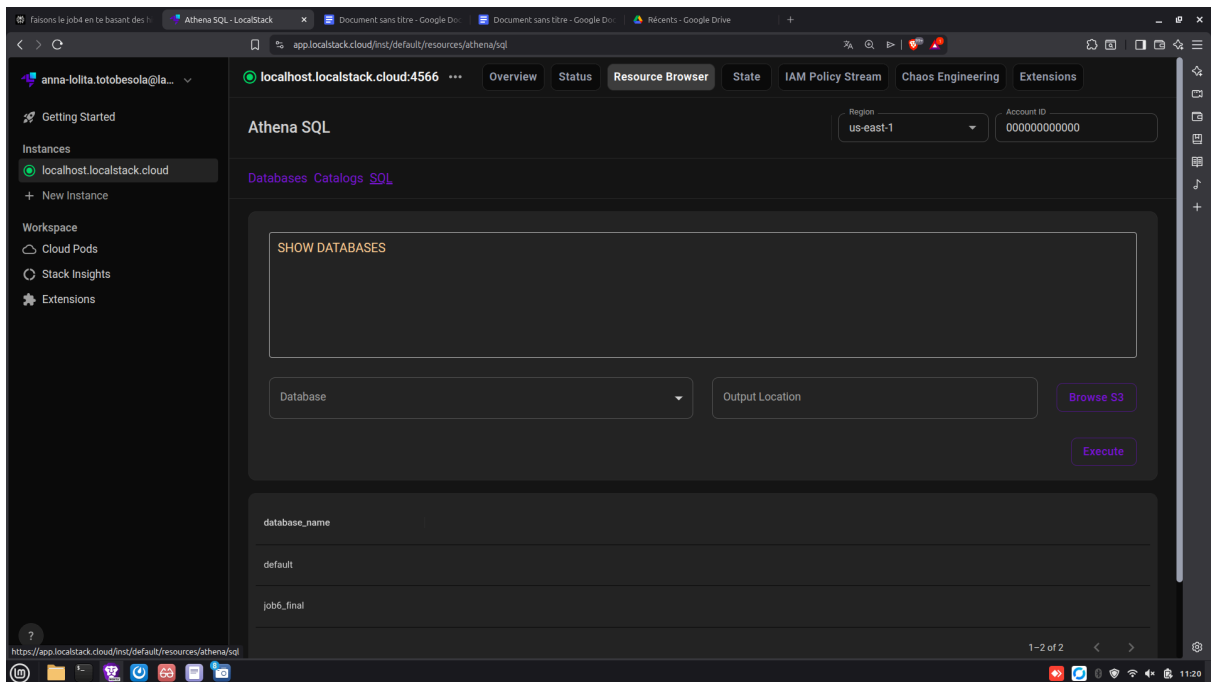
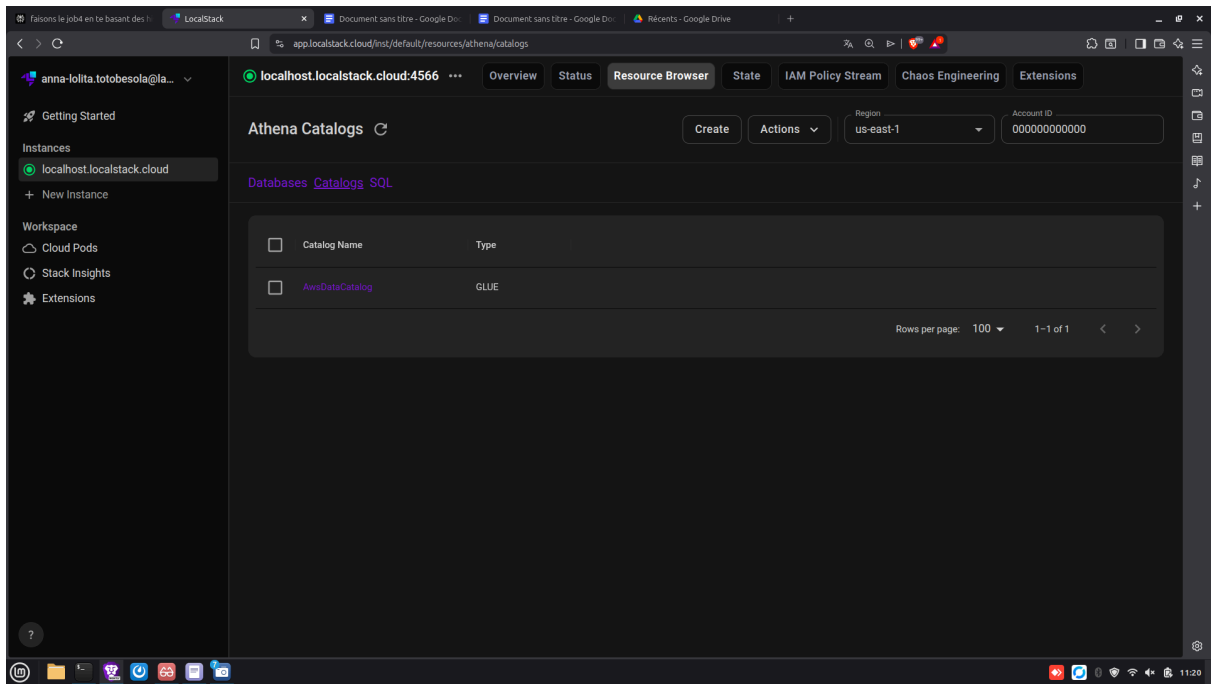
Athena / job6\_final

Databases Catalogs SQL

Table "sales\_input" schema

col_name	data_type	comment
user_id	string	
product	string	
amount	double	
status	string	

1-4 of 4



Résultat : Analyse de données fonctionnelle, rapports générés, compréhension de l'analytics.

### **JOB8 : Application conteneurisée**

ECS: service d'orchestration de conteneurs permettant de faire évoluer et de gérer des conteneurs Dockers sur un cluster.

Fargate : moteur de calcul pour l'exécution des conteneurs

ECR (Elastic Container Registry): Permet de stocker, gérer et déployer des images privées de conteneurs

```
``bash
```

```
1. Créer un repository ECR
```

```
awslocal ecr create-repository --repository-name my-app
```

```
2. Obtenir l'URI du repository
```

```
REPO_URI=$(awslocal ecr describe-repositories \
 --repository-name my-app \
 --query 'repositories[0].repositoryUri' --output text)
```

```
3. Créer une image Docker simple
```

```
cat > Dockerfile << EOF
```

```
FROM nginx:alpine
```

```
COPY index.html /usr/share/nginx/html/
```

```
EOF
```

```
echo '<h1>App from LocalStack ECR!</h1>' > index.html
```

```
4. Build et tag
```

```
docker build -t my-app .
```

```
docker tag my-app:latest $REPO_URI:latest
```

```
5. Push vers ECR LocalStack
```

```
Note: LocalStack ECR accepte les push sans authentication
```

```
docker tag my-app:latest localhost:4566/my-app:latest
```

```
6. Créer un cluster ECS (si Pro)
```

```
awslocal ecs create-cluster --cluster-name my-cluster
```

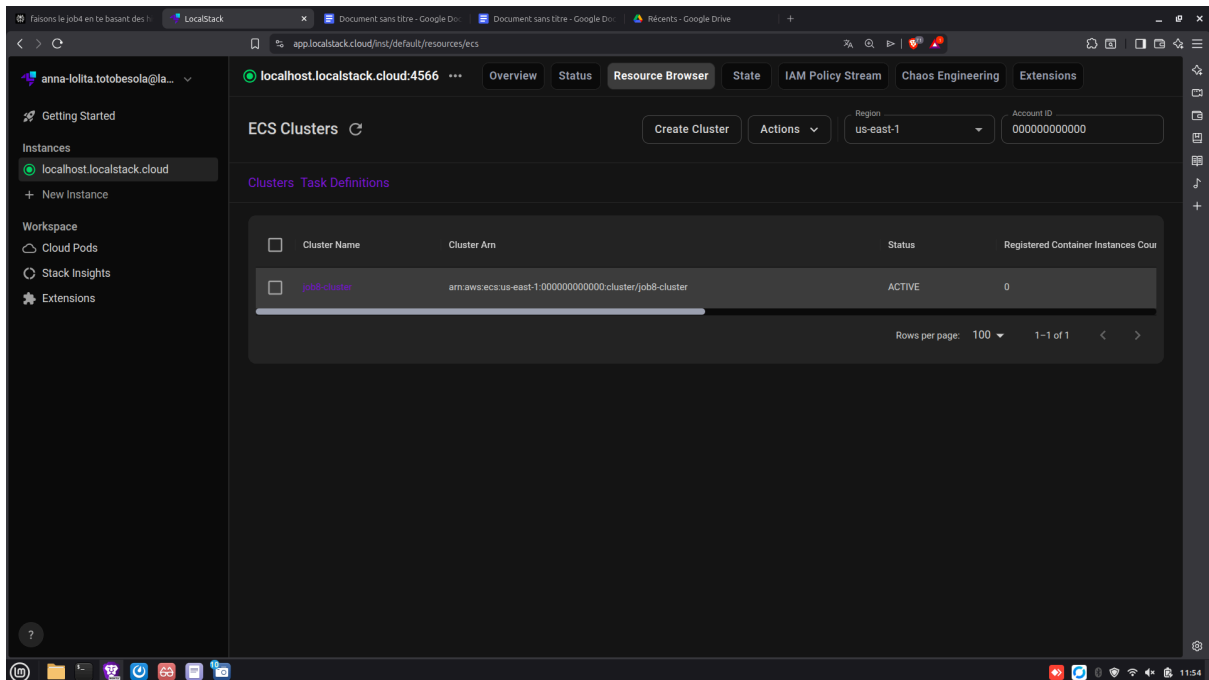
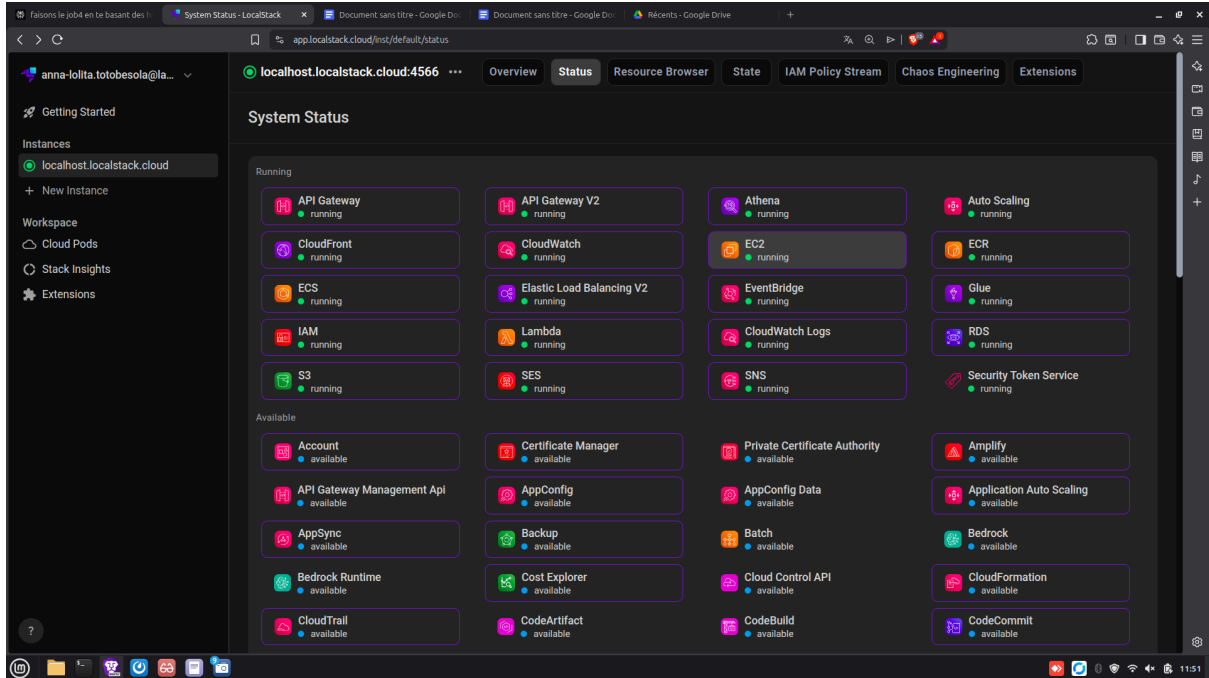
```
7. Enregistrer une task definition
```

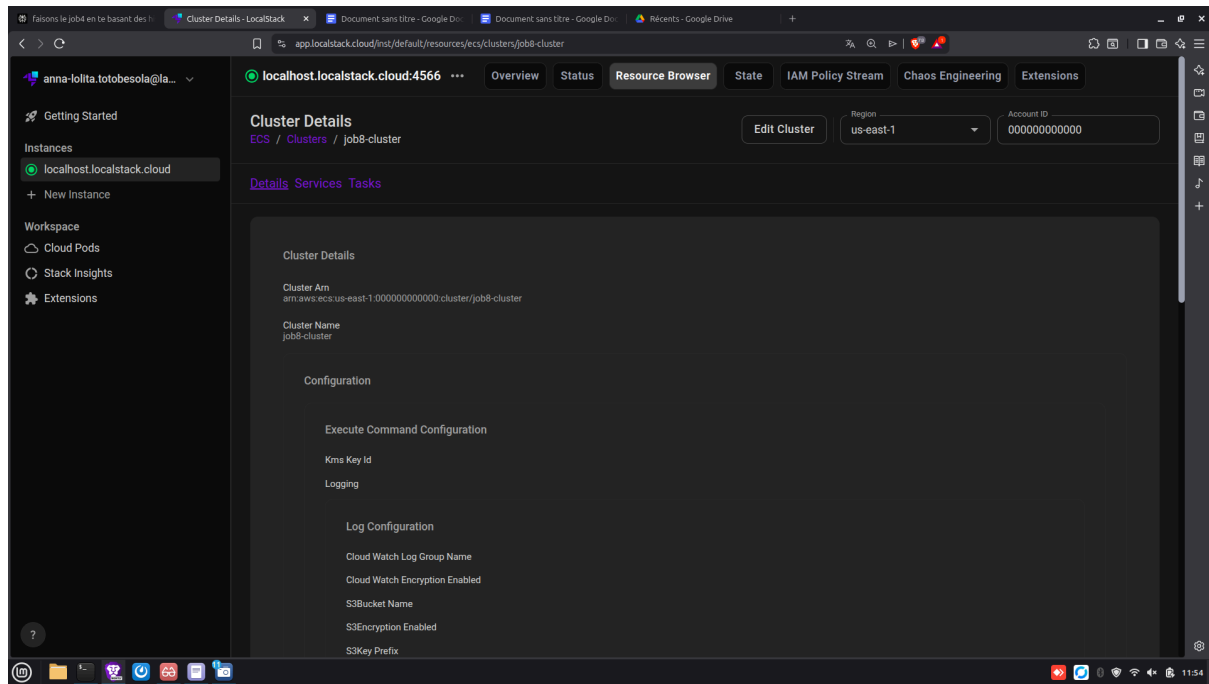
```
cat > task-def.json << EOF
```

```
{
 "family": "my-app-task",
 "networkMode": "bridge",
 "containerDefinitions": [{
 "name": "my-app-container",
 "image": "$REPO_URI:latest",
 "memory": 512,
 "portMappings": [{
 "containerPort": 80,
 "hostPort": 8080
 }]
 }]
}
```

```
}
EOF
```

```
awslocal ecs register-task-definition --cli-input-json file://task-def.json
```





### **Job9: Orchestration Serverless**

*Step Functions: service cloud permettant aux développeurs de créer des workflows pour les processus critiques de l'entreprise*

```
```bash
# Créer les fonctions Lambda du workflow
# Fonction 1: Validation
cat > validate.py << EOF
def lambda_handler(event, context):
    order = event.get('order', {})
    return {'valid': True, 'order': order}
EOF

zip validate.zip validate.py
awslocal lambda create-function \
    --function-name ValidateOrder \
    --runtime python3.11 \
    --role arn:aws:iam::000000000000:role/lambda-role \
    --handler validate.lambda_handler \
    --zip-file fileb://validate.zip

# Fonction 2: Processing
cat > process.py << EOF
def lambda_handler(event, context):
    return {'status': 'processed', 'order': event['order']}
EOF

zip process.zip process.py
awslocal lambda create-function \
```

```

--function-name ProcessOrder \
--runtime python3.11 \
--role arn:aws:iam::000000000000:role/lambda-role \
--handler process.lambda_handler \
--zip-file fileb://process.zip

# Créer la State Machine
cat > workflow.json << EOF
{
  "Comment": "Order Processing Workflow",
  "StartAt": "ValidateOrder",
  "States": {
    "ValidateOrder": {
      "Type": "Task",
      "Resource":
"arn:aws:lambda:us-east-1:000000000000:function:ValidateOrder",
      "Next": "ProcessOrder"
    },
    "ProcessOrder": {
      "Type": "Task",
      "Resource":
"arn:aws:lambda:us-east-1:000000000000:function:ProcessOrder",
      "End": true
    }
  }
}
EOF

awslocal stepfunctions create-state-machine \
  --name OrderWorkflow \
  --definition file://workflow.json \
  --role-arn arn:aws:iam::000000000000:role/stepfunctions-role

# Exécuter le workflow
awslocal stepfunctions start-execution \
  --state-machine-arn
arn:aws:states:us-east-1:000000000000:stateMachine:OrderWorkflow \
  --input '{"order": {"id": "123", "total": 99.99}}'

```

